UNIVERSIDADE ABERTA
www.uab.pt

Projeto Final em Engenharia Informática

Meal Planner

Gerador de Receitas e Plano Semanal de Refeições

RELATÓRIO FINAL

Mário Prazeres - 2203127
Matilde Carmo - 2201036

Pedro Pestana

Junho 2025

Índice

Lista de Figuras 4
Lista de Tabelas 4
Resumo 5
Objetivos 5
Capítulo 1 – Introdução 7
1.1 Ferramentas e Tecnologias Utilizadas 7
Capítulo 2 - Desenho 9
2.1. Mapa Gantt E Distribuição De Tarefas 9
2.2. Desenho Das Páginas Do Website 9
2.3 Schema 10
Tabela 2.3.1 – Estrutura da tabela users 11
Tabela 2.3.2 – Estrutura da tabela meal_plans 12
Tabela 2.3.3 – Estrutura da tabela meal_entries 12
Tabela 2.3.4 – Estrutura da tabela categories 13
Tabela 2.3.5 – Estrutura da tabela recipes 13
Tabela 2.3.6 – Estrutura da tabela blocked_recipes 15
Tabela 2.3.7 – Estrutura da tabela favorites 15
2.4 Base De Dados / API 15
Capítulo 3 - Implementação 17
3.1 Funcionalidades desenvolvidas 17
3.2 Ilustração das Funcionalidades Implementadas 18
Figura 3.2.1 – Formulário de Login 19
Figura 3.2.2 – Formulário de Registo 20
Figura 3.2.3 – Página inicial do Utilizador Comum 21
Figura 3.2.4 – Página inicial do Administrador 22
Figura 3.2.5 – Pesquisa de Receitas com Filtros 23
Figura 3.2.6 – Criação de Plano Semanal 24
Figura 3.2.7 – Detalhe da receita 25
Figura 3.2.8 – Submissão de receita 26
Figura 3.2.9 – Gestão de Utilizadores (Admin) 27
Figura 3.2.10 – Gestão de Categorias(Admin) 28
Figura 3.2.11 – Favoritos e Receitas Bloqueadas (Perfil do Utilizador) 29
Figura 3.2.12 – Página de Aprovação de Receitas (Admin) 30
Figura 3.2.13 – Página de edição de plano semanal 31
Figura 3.2.14 – Exportação de Plano Semanal em PDF 32
3.3 Manual de utilização 33
3.3.1 Registo e Autenticação 33
3.3.2 Pesquisa e Exploração de Receitas 33
3.3.3 Visualização de Receita 33

3.3.4 Submissão de Receitas 33
3.3.5 Gestão de Favoritos e Receitas Bloqueadas 33
3.3.6 Criação e Gestão de Planos Semanais 33
3.3.7 Funcionalidades de Administração 34
Capítulo 4 - Resultados Obtidos 35
4.1 Funcionalidades Ativas 35
4.1.1 Tabela de funcionalidades 35
4.2 Integração entre Componentes 36
4.3 Dados Atuais na Base de Dados 36
4.4 Validação Manual e automático das Funcionalidades 36
Figura 4.4.1 Resultado dos testes automáticos 38
Capítulo 5 - Conclusão 40
Bibliografia 42
Anexos 43
Anexo I - Diagrama da Base de Dados 43
Figura A1: Diagrama relacional da base de dados do Meal Planner 43
Anexo II – Código SQL de Criação das Tabelas 44
Tabela: users 44
Tabela: categories 44
Tabela: recipes 44
Tabela: meal_plans 45
Tabela: meal_entries 45
Tabela: favorites 45
Tabela: blocked_recipes 46
Anexo III – Estrutura de Diretórios e Ficheiros do Projeto Meal Planner 46
Estrutura de diretórios: 46
Anexo IV – Exemplo de Plano Semanal 48
1. Modelo SQLAlchemy (models/meal_plan.py) 48
2. Formulário WTForms (forms/meal_plan_form.py) 49
3. Rota Flask (routes/meal_plans.py) 49
4. Template HTML (templates/meal_plans/criar.html) 50
Anexo V – Modelos SQLAlchemy 50
User (models/user.py) 51
Recipe (models/recipe.py) 51
MealPlan & MealEntry (models/meal_plan.py) 52
Category (models/category.py) 53
Favorites (models/favorites.py) 53
BlockedRecipe (models/blocked_recipe.py) 54
Anexo VI – Validação com WTForms 54
1. Formulário de Registo (forms/register_form.py) 54
2. Formulário de Submissão de Receita (forms/recipe_form.py) 55

3. Formulário de Login (forms/login_form.py) 55
Anexo VII – Submissão de Receita **55**
1. Rota Flask (routes/recipes.py) 56
2. Template HTML (templates/recipes/submeter.html) 56
Anexo VIII – Autenticação **57**
1. Rota de Login (routes/auth.py) 57
2. Rota de Logout (routes/auth.py) 58
3. Métodos de Autenticação (models/user.py) 58
4. Gestão de Sessão e Acesso 58
Anexo IX – Gestão de Utilizadores **59**
1. Níveis de Utilizador 59
2. Rota Flask – Listagem de Utilizadores (routes/admin.py) 59
3. Rota Flask – Edição de Utilizador (routes/admin.py) 60
4. Template HTML – Listagem Visual (templates/admin/utilizadores.html) 60
Anexo X – Gestão de Categorias **61**
1. Rotas de Administração (routes/admin.py) 61
2. Visualização de Categorias (routes/categories.py) 62
3. Templates (categories/index.html e ver.html) 62

Lista de Figuras

Figura 2.1.1 – Mapa Gantt De Planeamento Do Projeto..... Página 9
Figura 2.2.1 – Visualização inicial do site..... Página 10
Figura 3.2.1 – Formulário de Login..... Página 19
Figura 3.2.2 – Formulário de Registo..... Página 20
Figura 3.2.3 – Página inicial do Utilizador Comum..... Página 21
Figura 3.2.4 – Página inicial do Administrador..... Página 22
Figura 3.2.5 – Pesquisa de Receitas com Filtros..... Página 23
Figura 3.2.6 – Criação de Plano Semanal..... Página 24
Figura 3.2.7 - Detalhe da Receita..... Página 25
Figura 3.2.8 - Submissão de Receita..... Página 26
Figura 3.2.9 - Gestão de utilizadores (Admin)..... Página 27
Figura 3.2.10 - Gestão de categorias (Admin)..... Página 28
Figura 3.2.11 - Favoritos e Receitas bloqueadas (Perfil de Utilizador..... Página 29
Figura 3.2.12 - Página de aprovação de receitas (Admin)..... Página 30
Figura 3.2.13 - Página de edição do plano semanal..... Página 31
Figura 3.2.14 - Exportação de plano semanal para PDF..... Página 32
Figura A1 – Diagrama relacional da base de dados Meal Planner..... Página 43

Lista de Tabelas

Tabela 2.3.1 – Estrutura da tabela users..... Página 11
Tabela 2.3.2 – Estrutura da tabela meal_plans..... Página 12
Tabela 2.3.3 – Estrutura da tabela meal_entries..... Página 12
Tabela 2.3.4 – Estrutura da tabela categories..... Página 13
Tabela 2.3.5 – Estrutura da tabela recipes..... Página 13
Tabela 2.3.6 – Estrutura da tabela blocked_recipes..... Página 15
Tabela 2.3.7 – Estrutura da tabela favorites..... Página 15
Tabela 4.1.1 - Tabela de funcionalidades..... Página 35

Resumo

Este relatório descreve o desenvolvimento completo do *Meal Planner*, uma aplicação web concebida para auxiliar os utilizadores na gestão personalizada de receitas e na elaboração de planos alimentares semanais. Ao longo do projeto, foram definidas e implementadas as principais funcionalidades da aplicação, incluindo o registo e autenticação de utilizadores, a criação e edição de receitas e a organização de planos semanais. O desenvolvimento envolveu a definição de uma estrutura de base de dados eficiente, a implementação de uma interface gráfica intuitiva, a adoção de tecnologias modernas de desenvolvimento web e a elaboração de testes. O relatório detalha ainda os desafios técnicos enfrentados e as soluções adotadas, bem como as decisões de design e arquitetura que orientaram o projeto. Em conclusão, a aplicação encontra-se funcional e estável, cumprindo os objetivos inicialmente propostos e oferecendo uma base sólida para futuras melhorias ou expansões.

Objetivos

O desenvolvimento do Meal Planner foi orientado por um conjunto de objetivos claros e alinhados com as necessidades identificadas na gestão alimentar do dia a dia. Pretendeu-se criar uma solução prática, intuitiva e flexível, capaz de facilitar a organização das refeições semanais e promover hábitos alimentares mais saudáveis e personalizados.
Para isso, definiram-se os seguintes objetivos principais:

- Desenvolver uma aplicação web responsiva que permita aos utilizadores pesquisar, registar e organizar receitas de forma eficiente;

- Implementar um sistema de planeamento semanal de refeições, possibilitando a seleção de receitas para cada dia e refeição da semana;

- Garantir a personalização da experiência do utilizador, através de filtros de pesquisa (ex: categoria alimentar, tempo de preparação), favoritos e receitas privadas;

- Assegurar a segurança e privacidade dos dados dos utilizadores, nomeadamente através de autenticação segura e encriptação de palavras-passe;

- Permitir a participação ativa dos utilizadores na plataforma, através da submissão de receitas, gestão de categorias e possibilidade de partilha pública ou manutenção privada das suas receitas;

- Criar um painel de administração que permita a gestão eficiente dos conteúdos, utilizadores e categorias, assegurando o correto funcionamento da plataforma;

- Promover a escalabilidade e manutenção futura do projeto, através da adoção de boas práticas de desenvolvimento, estrutura modular e documentação adequada.

Capítulo 1 – Introdução

O projeto **Meal Planner** tem como objetivo o desenvolvimento de uma aplicação web que permita aos utilizadores pesquisar, organizar e planear as suas refeições de forma simples, personalizada e eficiente. Através de filtros como tempo de preparação, categoria alimentar (vegetariana, saudável, etc.), entre outros, a aplicação facilita a descoberta de receitas alinhadas com as preferências e necessidades de cada utilizador.

Além da pesquisa, a aplicação disponibiliza uma funcionalidade de criação de plano semanal, permitindo ao utilizador selecionar receitas para cada refeição dos sete dias da semana. Esta funcionalidade visa promover uma melhor organização alimentar e facilitar a preparação antecipada das refeições. O plano pode ser posteriormente editado, adaptando-se às circunstâncias ou preferências do utilizador.

A plataforma contempla ainda a possibilidade de submissão de receitas pelos próprios utilizadores, com opção de partilha pública ou manutenção privada, bem como a gestão de categorias e a administração dos conteúdos por parte de utilizadores com nível de acesso superior. É também possível marcar receitas como favoritas e ocultar receitas não pretendidas, personalizando ainda mais a experiência de utilização.

A concretização deste projeto só foi possível graças aos conhecimentos adquiridos ao longo da Licenciatura em Engenharia Informática. Em particular, destacam-se as unidades curriculares de Linguagens de Programação, que forneceram as bases essenciais para o desenvolvimento em diferentes ambientes e paradigmas; Fundamentos de Base de Dados, que permitiram estruturar e gerir eficientemente os dados da aplicação; e Gestão de Projetos em Informática, que ofereceu metodologias e ferramentas úteis para planear, organizar e acompanhar o progresso do projeto. Para além destas, outras unidades curriculares também contribuíram de forma significativa, refletindo-se nas várias fases de desenvolvimento e nas decisões técnicas tomadas ao longo do processo.

1.1 Ferramentas e Tecnologias Utilizadas

O desenvolvimento da aplicação recorreu a um conjunto de ferramentas e tecnologias modernas, escolhidas pela sua robustez, flexibilidade e facilidade de integração:

- **Python e Flask:** Linguagem de programação principal e micro-framework para construção do backend;
- **HTML, CSS e Jinja2:** Tecnologias utilizadas para a construção das interfaces visuais;
- **PostgreSQL:** Sistema de gestão de base de dados relacional;

- **pgAdmin 4:** Ferramenta gráfica utilizada para gestão, consulta e manutenção da base de dados PostgreSQL alojada na plataforma Railway.

- **SQLAlchemy:** ORM que permite o mapeamento entre os modelos de Python e as tabelas da base de dados;

- **WTForms:** Biblioteca para a criação e validação de formulários web;

- **Werkzeug:** Utilizada para hashing seguro das passwords;

- **Railway:** Plataforma de alojamento gratuita e prática, escolhida por permitir o deploy fácil tanto da base de dados PostgreSQL como da aplicação web. Outras soluções foram descartadas por implicarem custos elevados ou estarem mais orientadas para sites estáticos;

- **Git e GitHub:** Ferramentas de controlo de versões e repositório público de código, utilizadas para colaboração e histórico do projeto;

- **VS Code:** Ambiente de desenvolvimento utilizado por ambos os elementos da equipa.

O projeto Meal Planner foi desenvolvido por Mário Prazeres e Matilde Carmo, no âmbito da Licenciatura em Engenharia Informática. O repositório público encontra-se disponível no GitHub em:
https://github.com/luniballony/Meal-Planner
A versão atual da aplicação pode ser acedida online em:
https://web-production-4e3b6.up.railway.app

Capítulo 2 - Desenho

2.1. Mapa Gantt E Distribuição De Tarefas

Para organizar de forma eficiente o trabalho, recorreu-se aos conhecimentos adquiridos na unidade curricular de Gestão de Projetos Informáticos, elaborando um diagrama de Gantt — uma ferramenta clássica de gestão de projetos, que permite listar tarefas, estimar durações e identificar prioridades (Geraldi & Lechler, 2012; Davis & Yen, 2002).

A partir desta representação visual, foi possível identificar as dependências entre tarefas e estruturar a sequência ideal para a sua execução.

Figura 2.1.1 – Mapa Gantt De Planeamento Do Projeto

Para melhor visualização, foram removidas duas semanas de folga do cronograma original, mantendo-se as principais tarefas e respectiva distribuição temporal.

				17/03/2025	24/03/2025	31/03/2025	07/04/2025	14/04/2025	21/04/2025	28/04/2025	05/05/2025	12/05/2025	19/05/2025	26/05/2025	02/06/2025	0
Atividade	Atividade	Precendentes 'simplificados'	Duração Média	semana 1	semana 2	semana 3	semana 4	semana 5	semana 6	semana 7	semana 8	semana 9	semana 10	semana 11	semana 12	se
A2	analisar api disponiveis	-	1 semana													
A3	desenhar a schema	A2	1 semana													
A4	criar um esboço do design do site	-	1 semana													
A5	criar base de dados	A3	2 semanas													
A6	criar estrutura inicial de ficheiros	-	1 semana													
A7	implementar autenticação	A6	1 semana													
A8	testar BD	A5	1 semana													
A9	criar estrutura inicial de html	A6	1 semana													
A10	implementar pesquisa e filtros	A9	3 semanas													
A11	criar página de receitas individuais	A9	2 semanas													
A12	criar página do plano semanal	A9	2 semanas													
A13	ligar frontend ao backend	A4, A10, A11, A12	2 semanas													
A14	implementar fav + ocultar receitas	A9, A8	1 semana													
A15	implementar/melhorar css	A14, A13	2 semanas													
A16	testar em diferentes dimensões de ecrã	A10, A15	1 semanas													
A17	testar o site em vários dispositivos	A16	1 semanas													
A18	testes + corrigir bugs	A17	1 semanas													
A19	criar página autenticação	A7	1 semana													

Em relação à distribuição de tarefas, sendo a equipa composta por dois elementos, foi acordado que ambos colaborariam em todas as etapas do projeto, contribuindo ativamente para o seu desenvolvimento. No entanto, verificou-se uma divisão natural de responsabilidades: o colega Mário Prazeres concentrou-se principalmente na componente de backend, relacionada com a gestão e manutenção de dados, enquanto a colega Matilde Carmo se focou maioritariamente no desenvolvimento da interface e estrutura do website (frontend).

2.2. Desenho Das Páginas Do Website

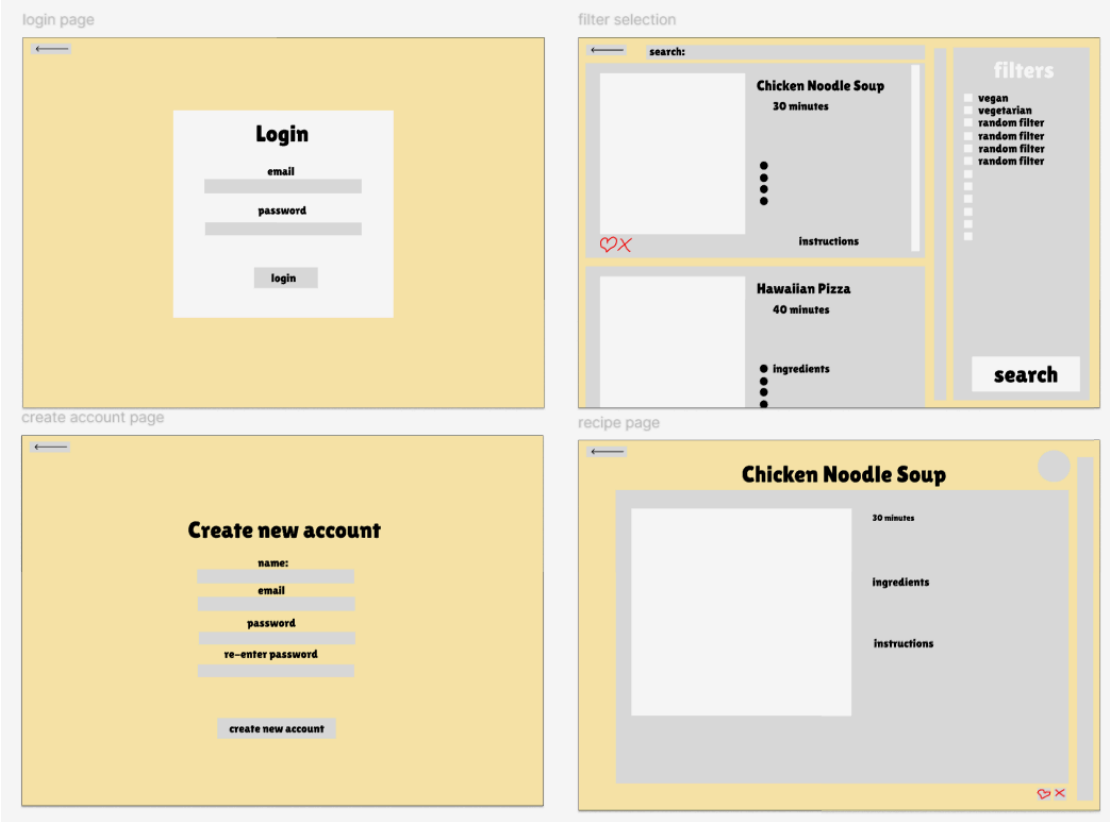
Para facilitar a implementação da aplicação, foi desenvolvido um esboço gráfico das várias páginas do website, recorrendo à ferramenta Figma. Este protótipo inicial, criado nas primeiras semanas do projeto, permitiu visualizar de forma clara a estrutura e a estética pretendidas para a aplicação.

A existência deste modelo gráfico facilitou significativamente a transição para a implementação em HTML e CSS, ao fornecer uma base sólida para o layout e o estilo das páginas.

Além disso, o esboço permitiu uma estimativa mais rigorosa do volume de trabalho envolvido, possibilitando contabilizar o número de páginas a desenvolver, planear os respetivos conteúdos e antecipar as funcionalidades que cada uma deveria suportar.

Abaixo apresentam-se exemplos de algumas das páginas desenhadas.

2.2.1 - Figura da visualização inicial do site



2.3 Schema

Paralelamente, e com base nos conceitos de modelação de dados adquiridos ao longo do curso — em particular na disciplina de Fundamentos de Base de Dados —, foi desenvolvida a *schema* da base de dados (ver Anexo I – Diagrama da Base de Dados - Figura A1), considerada uma das tarefas prioritárias na fase inicial do projeto.

A estrutura da base de dados foi concebida de forma a refletir as principais entidades da aplicação, sendo especialmente relevante a definição das tabelas 'User' e 'Receita'. A tabela 'User' determina os campos necessários durante o registo de um novo utilizador, enquanto a tabela 'Receita' estrutura a informação fundamental associada a cada receita, como ingredientes, instruções de preparação, dificuldade, entre outros.

O desenvolvimento antecipado da schema revelou-se crucial para orientar tanto a criação da base de dados como a lógica de funcionamento de várias funcionalidades da aplicação, como a filtragem de receitas e o planeamento semanal. Esta abordagem facilitou igualmente uma integração mais fluida entre o frontend e o backend.

A importância de um design cuidadoso da base de dados é amplamente reconhecida na literatura, sendo essencial para garantir a integridade dos dados e a eficiência do sistema (Burkett, 1997; Peinl, 2021).

Tabela 2.3.1 – Estrutura da tabela users

A tabela `users` armazena a informação relativa a cada utilizador registado na aplicação. Inclui dados essenciais para autenticação, controlo de permissões e estado de atividade da conta. Cada utilizador possui um nível de acesso que define o seu perfil: utilizador comum, premium ou administrador.

Campo	Tipo de Dados	Descrição (automática)
id	INTEGER	Identificador único do utilizador
nome	VARCHAR(100)	Nome do utilizador
email	VARCHAR(120)	Endereço de email do utilizador
password_hash	VARCHAR(255)	Palavra-passe cifrada com algoritmo seguro (scrypt), nunca armazenando a password original em texto simples.
nivel	INTEGER	Nível de acesso (1: comum, 2: premium, 3: administrador)
criado_em	DATETIME	Data e hora da criação da conta
ativo	BOOLEAN	Estado da conta (1: ativa, 0: bloqueada ou inativa)

Tabela 2.3.2 – Estrutura da tabela meal_plans

A tabela `meal_plans` regista os planos semanais criados pelos utilizadores. Cada plano corresponde a uma semana de refeições e está associado a um único utilizador. A sua ligação às refeições específicas é feita através da tabela `meal_entries`.

Campo	Tipo de Dados	Descrição (automática)
id	INTEGER	Identificador único do plano semanal
data_inicio	DATE	Corresponde ao primeiro dia do plano semanal, a partir do qual as refeições são planeadas para os sete dias seguintes.
criado_em	DATETIME	Data e hora em que o plano foi criado
utilizador_id	INTEGER	Referência ao utilizador que criou o plano (users)

Tabela 2.3.3 – Estrutura da tabela meal_entries

A tabela `meal_entries` tem como função armazenar cada refeição individual associada a um plano semanal. Esta estrutura permite que um plano (`meal_plans`) seja composto por várias refeições, organizadas por data e tipo (pequeno-almoço, almoço ou jantar). Cada entrada associa uma determinada receita a uma refeição específica de um dia.

Esta abordagem permite uma grande flexibilidade na criação dos planos, tornando possível, por exemplo, planear apenas os jantares da semana, ou definir diferentes combinações de refeições ao longo dos dias.

Campo	Tipo de Dados	Descrição (automática)
id	INTEGER	Identificador único da entrada no plano
data_refeicao	DATE	Data em que a refeição está planeada
refeicao_tipo	VARCHAR(50)	Tipo de refeição (Pequeno almoço, almoço ou jantar)

plano_id	INTEGER	Referência ao plano semanal (meal_plans) ao qual esta entrada pertence
receita_id	INTEGER	Referência à receita associada a esta refeição (recipes)

Tabela 2.3.4 – Estrutura da tabela categories

A tabela `categories` organiza as receitas por tipo, permitindo aplicar filtros e facilitar a pesquisa dentro da aplicação. Cada categoria representa um grupo alimentar ou temática culinária (ex: vegetariano, saladas, carne). Esta classificação é usada tanto na submissão de receitas como na sua visualização.

Campo	Tipo de Dados	Descrição (automática)
id	INTEGER	Identificador único da categoria
nome	VARCHAR(100)	Nome da categoria (ex: Saladas, Vegetariano, Carne, etc.)
descricao	TEXT	Descrição da categoria (campo previsto, mas não utilizado nesta versão)

Tabela 2.3.5 – Estrutura da tabela recipes

A tabela `recipes` armazena todas as receitas disponíveis na aplicação, sejam elas públicas ou privadas. Cada receita contém informações detalhadas como ingredientes, instruções, tempo de preparação, dificuldade e tags. Está também associada a um utilizador e a uma categoria, podendo ser submetida para aprovação antes de ficar visível publicamente.

Campo	Tipo de Dados	Descrição (automática)
id	INTEGER	Identificador único da receita
titulo	VARCHAR(150)	Título da receita
descricao	TEXT	Breve descrição da receita

ingredientes	TEXT	Lista dos ingredientes utilizados na receita
instrucoes	TEXT	Instruções passo a passo
tempo_preparacao	INTEGER	Tempo estimado da preparação (em minutos)
dificuldade	INTEGER	Grau de dificuldade (1: fácil, 2: médio, 3: difícil)
tags	VARCHAR	Palavras chave associadas à receita
data_submetida	TIMESTAMP	Data e hora de submissão
publicada	BOOLEAN	Indica se a receita está publicada (1) ou não (0)
fonte	VARCHAR	Fonte original da receita (ainda não implementado). (ex: meal planner)
utilizador_id	INTEGER	Referência ao utilizador (users) que criou a receita
categoria_id	INTEGER	Referência à categoria (categories) associada à receita
aprovada	BOOLEAN	Indica se a receita foi aprovada pelo administrador (1)
publica_quando_aprovada	BOOLEAN	Se marcada, a receita torna-se pública após aprovação (1)

Tabela 2.3.6 – Estrutura da tabela blocked_recipes

A tabela `blocked_recipes` regista as receitas que cada utilizador decidiu ocultar da sua visualização na aplicação. Esta funcionalidade permite personalizar a experiência do utilizador, evitando que lhe sejam sugeridas receitas que não pretende ver novamente.

Campo	Tipo de Dados	Descrição (automática)
utilizador_id	INTEGER (PK)	Referência ao utilizador que bloqueou a receita
receita_id	INTEGER (PK)	Referência à receita bloqueada

Tabela 2.3.7 – Estrutura da tabela favorites

A funcionalidade de receitas favoritas foi implementada com a tabela `favorites`, permitindo aos utilizadores marcar receitas para acesso rápido no seu perfil. Cada combinação de utilizador e receita corresponde a uma entrada nesta tabela.

Campo	Tipo de Dados	Descrição (automática)
utilizador_id	INTEGER (PK)	Referência ao utilizador que marcou a receita como favorita
receita_id	INTEGER (PK)	Referência à receita marcada como favorita

2.4 Base De Dados / API

A escolha de uma API revelou-se um ponto crucial para o desenvolvimento do projeto, uma vez que grande parte das funcionalidades dependia inicialmente dos dados fornecidos por fontes externas. As APIs são componentes essenciais nos ecossistemas de software modernos, permitindo a interação entre diferentes sistemas e facilitando a integração de funcionalidades (Manikas, 2016).

A estrutura da base de dados foi concebida tendo em conta o tipo de informação disponibilizada pelas APIs analisadas — como, por exemplo, o valor calórico das receitas, a sua origem, a indicação de serem vegetarianas ou não, e a qualidade e fiabilidade geral dos dados. Assim, a análise das possíveis APIs foi uma das primeiras tarefas realizadas no início do projeto.

No entanto, concluiu-se que nenhuma das opções disponíveis oferecia exatamente o que se pretendia, ou impunha limites de requisições diárias demasiado restritivas, o que inviabilizaria o funcionamento contínuo do website. Esta limitação é comum na integração de APIs, tornando a escolha adequada fundamental para garantir a eficiência e a escalabilidade do sistema (Bagga, 2023).

Face a este cenário, optou-se por criar uma base de dados própria e inserir manualmente as receitas. Embora este método exija mais tempo e trabalho, oferece total controlo sobre a informação armazenada, permite definir o nível de detalhe pretendido e elimina qualquer tipo de restrição no acesso aos dados. Esta abordagem também permite uma melhor adaptação às necessidades específicas do projeto, alinhando-se com as práticas recomendadas de design e especificação de APIs (Foster et al., 2023).

Em resumo, a decisão de manter uma base de dados autónoma revelou-se fundamental para garantir a flexibilidade, a fiabilidade e a personalização necessárias ao sucesso do Meal Planner.

Capítulo 3 - Implementação

O projeto foi desenvolvido utilizando uma abordagem web full-stack, integrando diferentes tecnologias para permitir a comunicação eficaz entre o frontend e o backend. O frontend foi implementado com recurso a HTML, CSS e Jinja2, proporcionando uma interface interativa e responsiva para o utilizador. O backend garante a gestão de dados e a lógica de negócio associada às funcionalidades do programa, comunicando com uma base de dados relacional desenvolvida em PostgreSQL.

Foi adotada uma estrutura modular, com separação clara entre as diferentes camadas do sistema: apresentação, lógica de aplicação e persistência de dados. Esta separação facilita a manutenção e a escalabilidade do sistema.

Esta abordagem modular assegura que o sistema é facilmente mantido, escalável e preparado para eventuais evoluções futuras.

3.1 Funcionalidades desenvolvidas

Foram implementadas diversas funcionalidades fundamentais para garantir o funcionamento completo da aplicação, tanto para utilizadores comuns como para administradores.

O sistema permite o **registo de novos utilizadores**, **login com validação de credenciais** e **logout seguro**. Com base nos conhecimentos adquiridos nas unidades curriculares de Segurança Em Redes E Em Computadores e Ética e Práticas em Engenharia, tornou-se evidente a importância de garantir a proteção dos dados sensíveis dos utilizadores, em particular das suas palavras-passe. Para evitar qualquer tipo de acesso indevido — seja externo ou por parte de administradores da plataforma — foi implementado um sistema de hashing, que armazena as palavras-passe na base de dados de forma encriptada. Esta abordagem segue as boas práticas recomendadas em termos de segurança e privacidade, assegurando que, mesmo em caso de violação da base de dados, as credenciais dos utilizadores permanecem protegidas (Tavani, 2023; Werkzeug Documentation, 2024).

Foram definidos três níveis distintos de utilizadores: **comum**, **premium** e **administrador**. A categoria premium já se encontra registada na base de dados, embora nesta fase ainda não possua diferenças funcionais face ao utilizador comum. Já o nível de administrador permite o acesso a funcionalidades exclusivas de gestão.

Após o login, o utilizador acede a uma **área personalizada**, com acesso à **criação de planos de refeições**, **pesquisa de receitas** e **submissão de novas receitas**. A funcionalidade de pesquisa permite aplicar filtros por categoria e tempo de preparação, sendo as receitas apresentadas com os principais detalhes visíveis, incluindo tempo estimado, grau de dificuldade, ingredientes e etiquetas (*tags*).

O sistema permite ainda a **submissão de novas receitas**, com a possibilidade de escolha entre publicação pública ou manutenção em modo privado. As receitas submetidas ficam em estado pendente até serem revistas e aprovadas por um administrador. O formulário de submissão inclui campos para o título, descrição, tempo de preparação, dificuldade, ingredientes, instruções detalhadas, categoria e tags.

A funcionalidade de **criação de plano semanal** permite ao utilizador selecionar, para cada dia da semana, uma receita distinta para pequeno-almoço, almoço e jantar. Estes planos podem ser posteriormente visualizados, editados, eliminados ou transferidos em formato PDF. Esta exportação está atualmente disponível apenas em modo local, devido a limitações da plataforma Railway, que não suporta a biblioteca necessária para ativar esta funcionalidade (Railway Documentation, 2024).

Ao nível da administração, o **painel do administrador** inclui um conjunto alargado de funcionalidades: **aprovação ou rejeição de receitas submetidas**, **gestão de categorias** (com possibilidade de criar, editar ou eliminar categorias) e **gestão de utilizadores** (incluindo bloqueio ou desbloqueio de contas, alteração do nível de acesso e visualização de dados).

Foi também implementada a funcionalidade de **favoritos e ocultação de receitas**, permitindo aos utilizadores adicionar receitas aos seus favoritos para mais facilmente as encontrarem, ou bloquear receitas para que deixem de ser sugeridas na navegação. Ambas as funcionalidades podem ser geridas a partir do perfil do utilizador e das páginas de receitas.

Todas estas funcionalidades são geridas num ambiente seguro e validado, respeitando as boas práticas de desenvolvimento de aplicações web, tanto ao nível da arquitetura como da gestão de dados e autenticação dos utilizadores. Foram seguidas recomendações de referência para a robustez, segurança e integridade do sistema, conforme preconizado por Silberschatz, Korth e Sudarshan (2020), bem como pelas principais bibliotecas e frameworks utilizadas ao longo do projeto.

3.2 Ilustração das Funcionalidades Implementadas

Este subcapítulo apresenta capturas de ecrã de algumas das páginas desenvolvidas da aplicação Meal Planner, com o objetivo de demonstrar visualmente o estado atual da implementação e reforçar as descrições das funcionalidades anteriormente apresentadas.

Figura 3.2.1 – Formulário de Login

A interface de autenticação permite ao utilizador introduzir as suas credenciais (email e palavra-passe) para aceder à área reservada da aplicação.

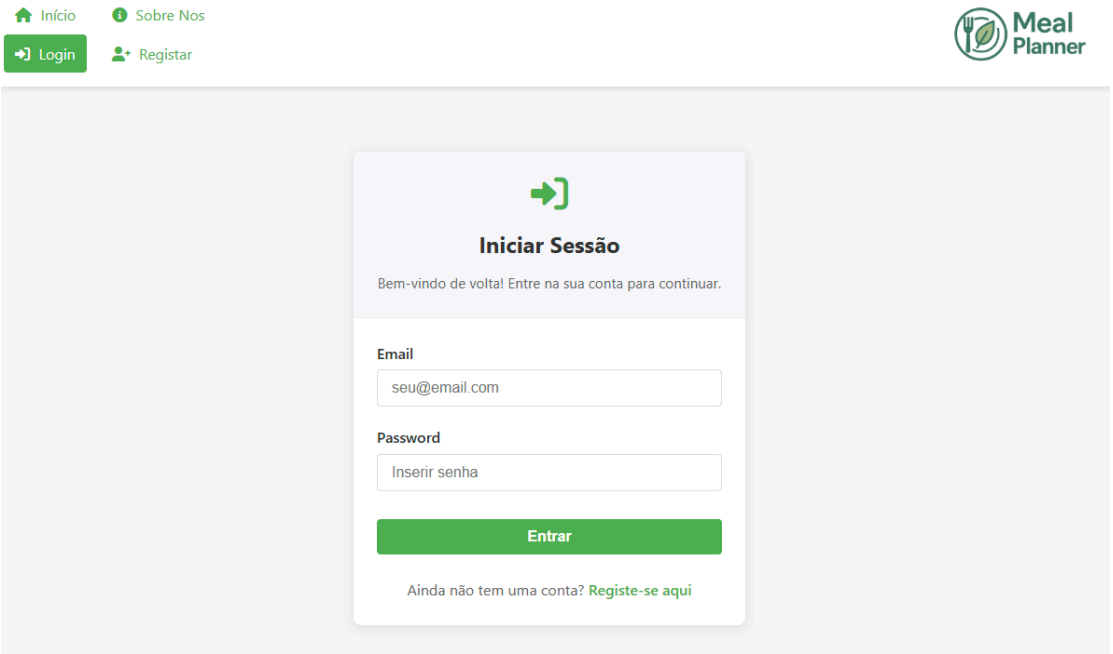


Figura 3.2.2 – Formulário de Registo

Permite a criação de uma nova conta, exigindo nome, email e definição de palavra-passe com confirmação.

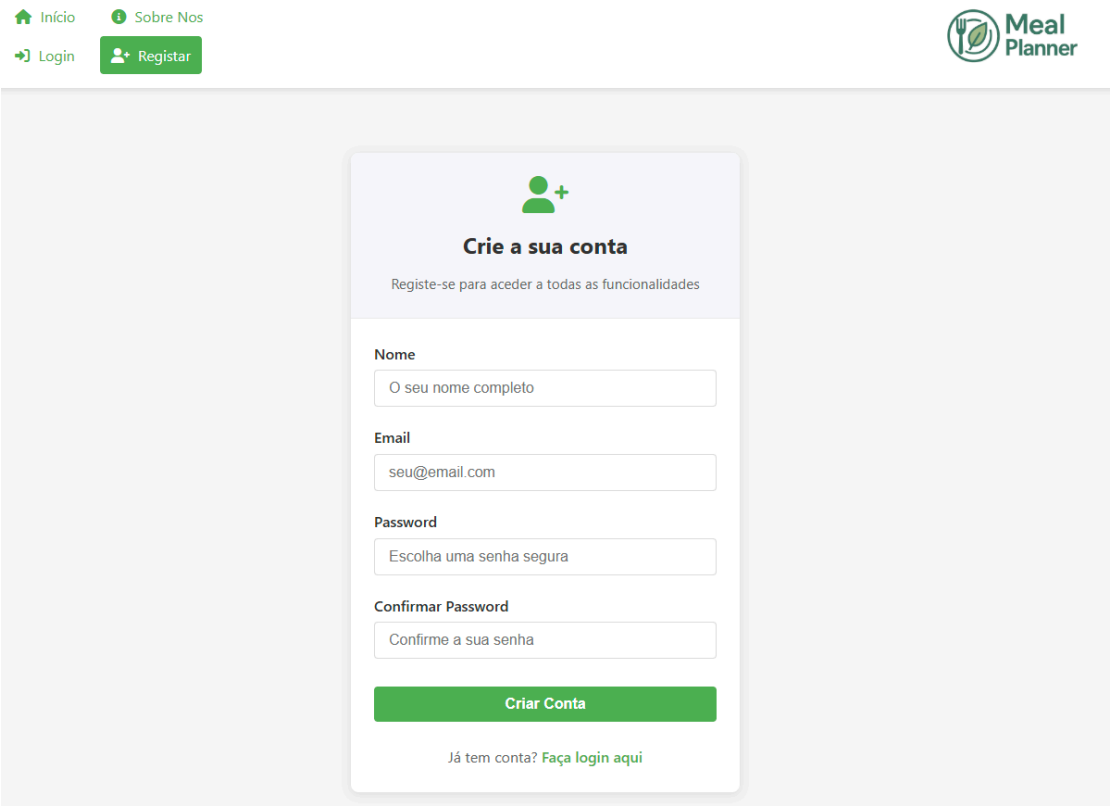


Figura 3.2.3 – Página inicial do Utilizador Comum

Após autenticação, o utilizador comum é direcionado para a sua área pessoal, onde pode aceder a receitas, criar planos de refeição, explorar categorias, ver as suas receitas favoritas e as bloqueadas.

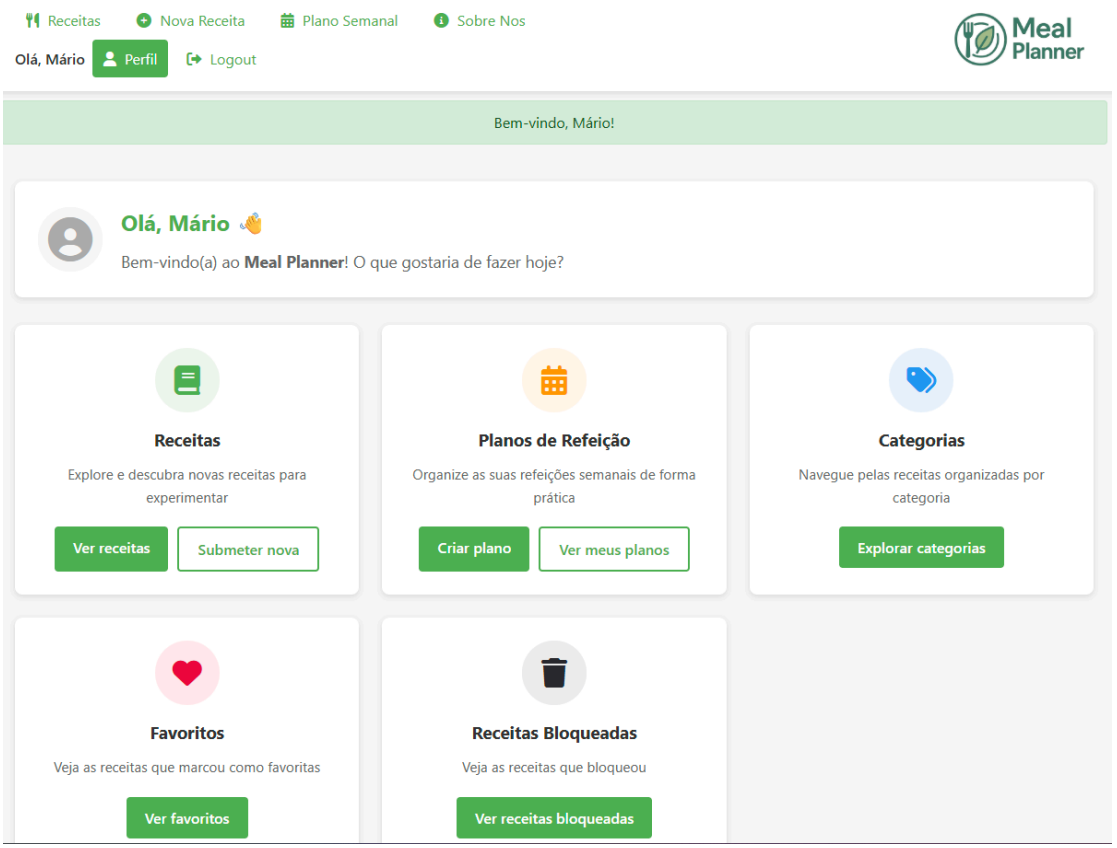


Figura 3.2.4 – Página inicial do Administrador

O administrador dispõe de um conjunto de opções adicionais no menu, incluindo a gestão de utilizadores, receitas, categorias e aprovação de receitas.

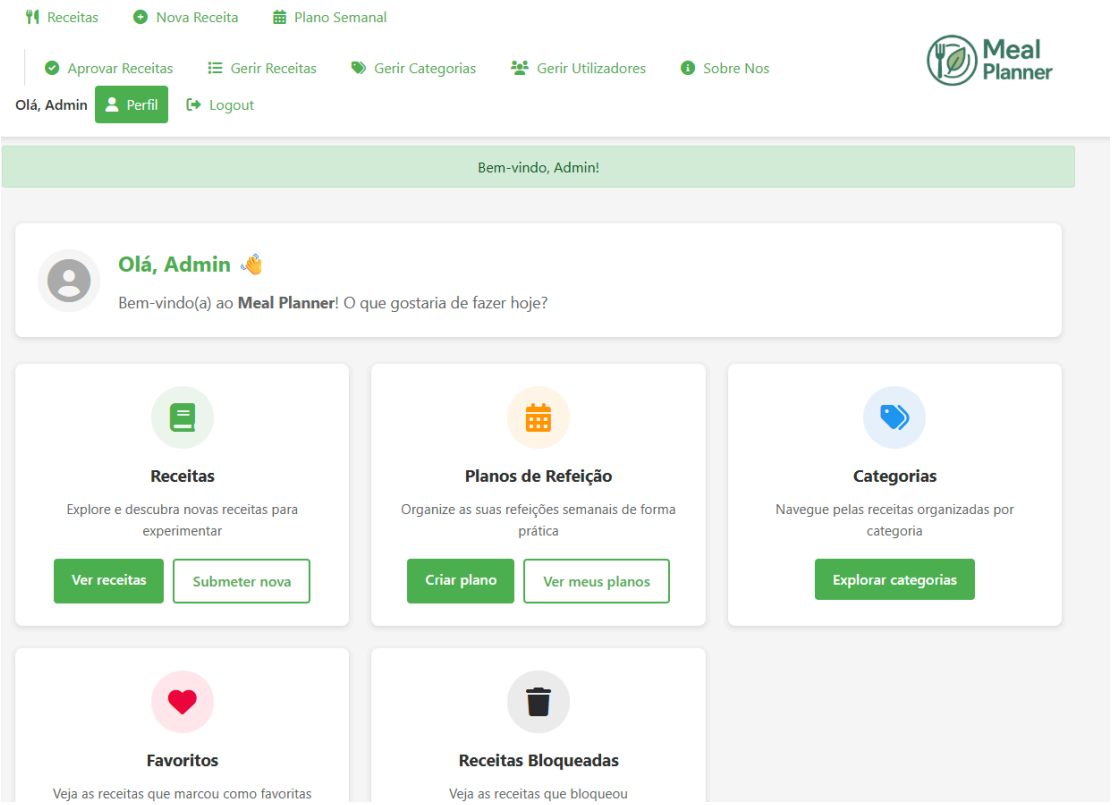


Figura 3.2.5 – Pesquisa de Receitas com Filtros

O utilizador pode procurar receitas ao aplicar filtros por categoria e tempo de preparação. As receitas são apresentadas com os detalhes mais relevantes.

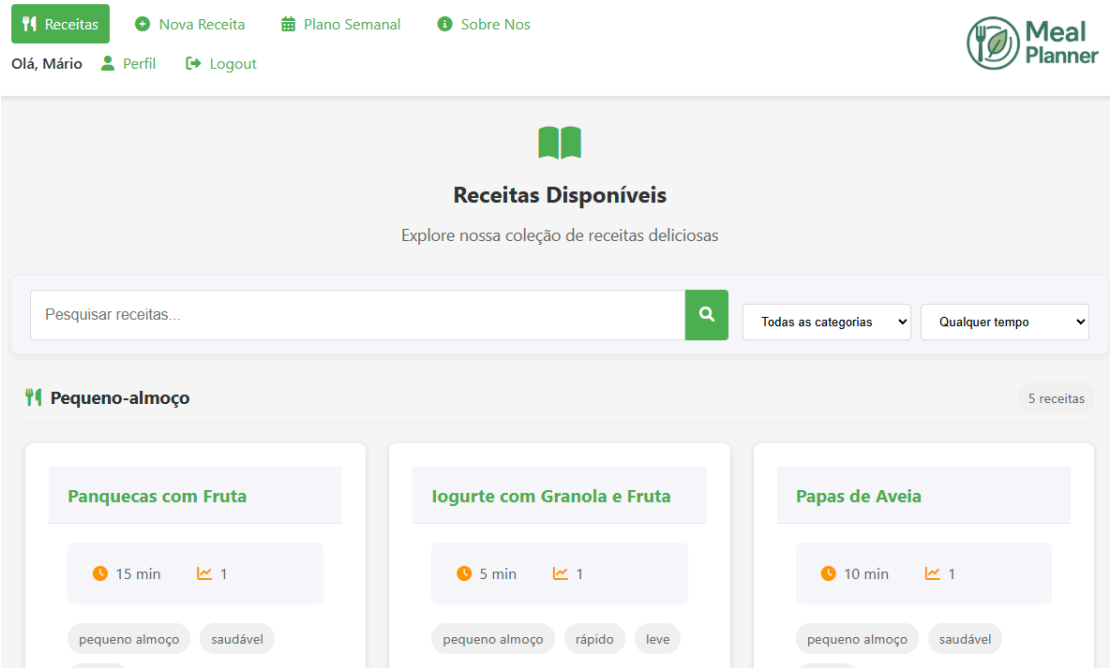


Figura 3.2.6 – Criação de Plano Semanal

Permite ao utilizador seleccionar, para cada refeição do dia, uma receita diferente, organizando de forma prática a sua semana alimentar.

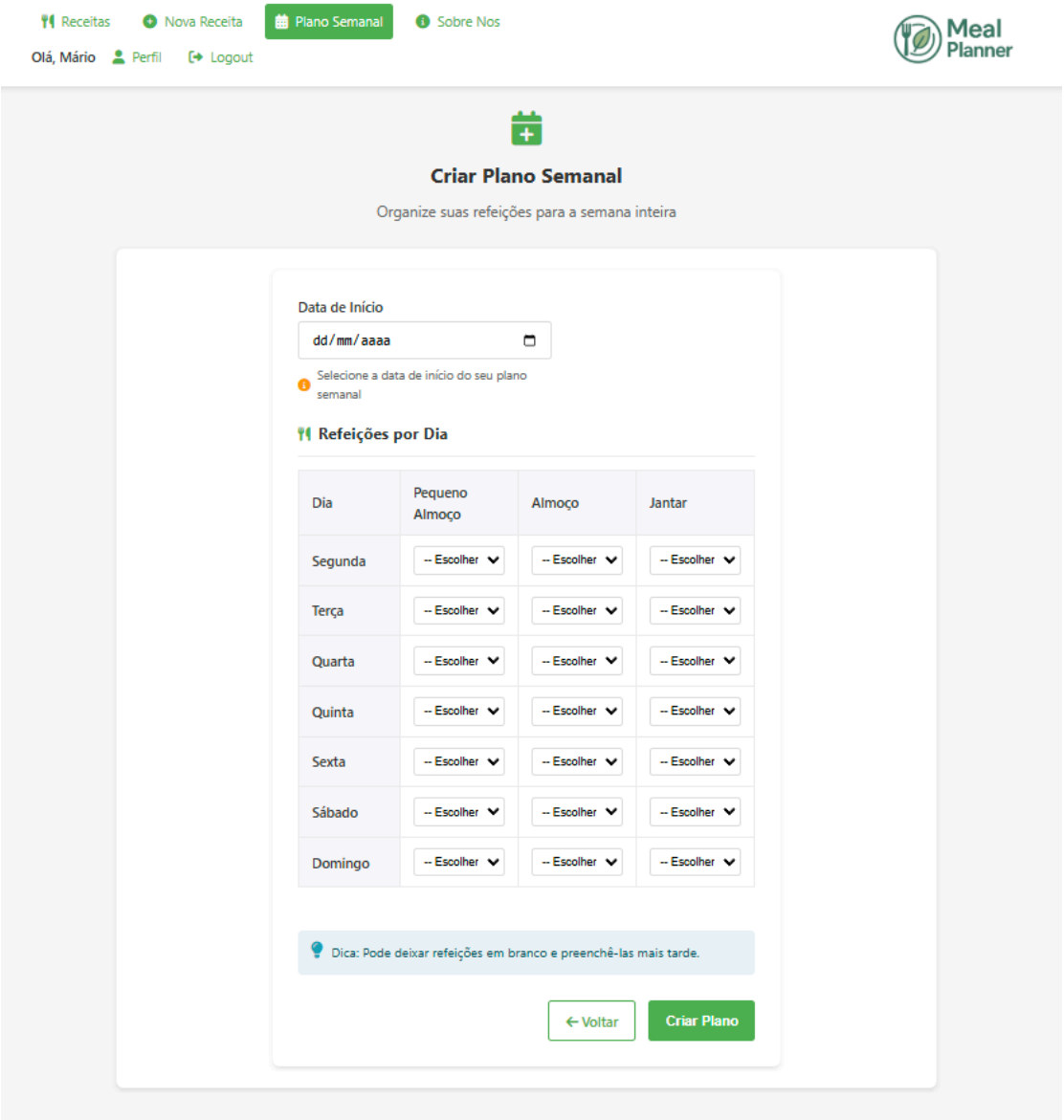


Figura 3.2.7 – Detalhe da receita

Mostra a visualização de uma receita individual (ingredientes, instruções, tags, botões favoritos/bloquear, etc.)

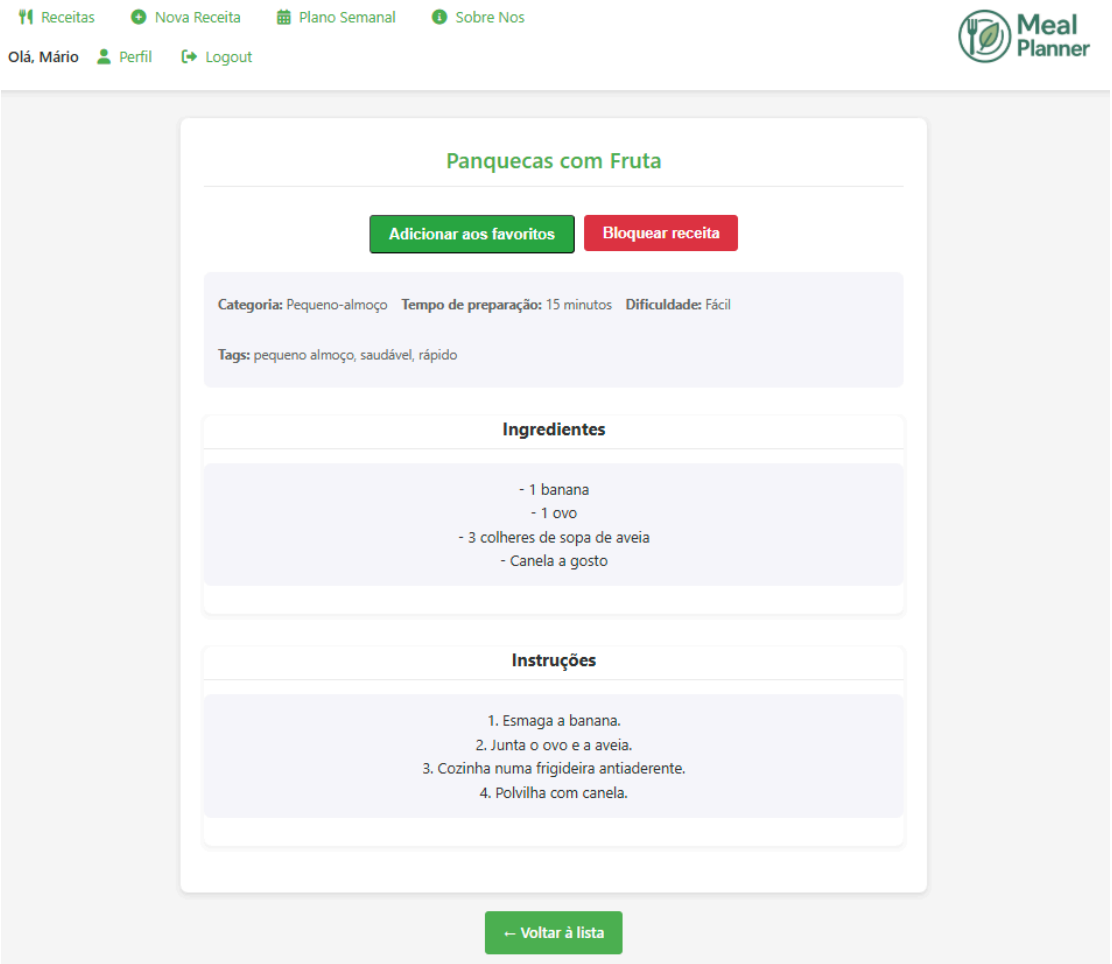


Figura 3.2.8 – Submissão de receita

Print do formulário de submissão de nova receita (com campos e opções pública/privada).

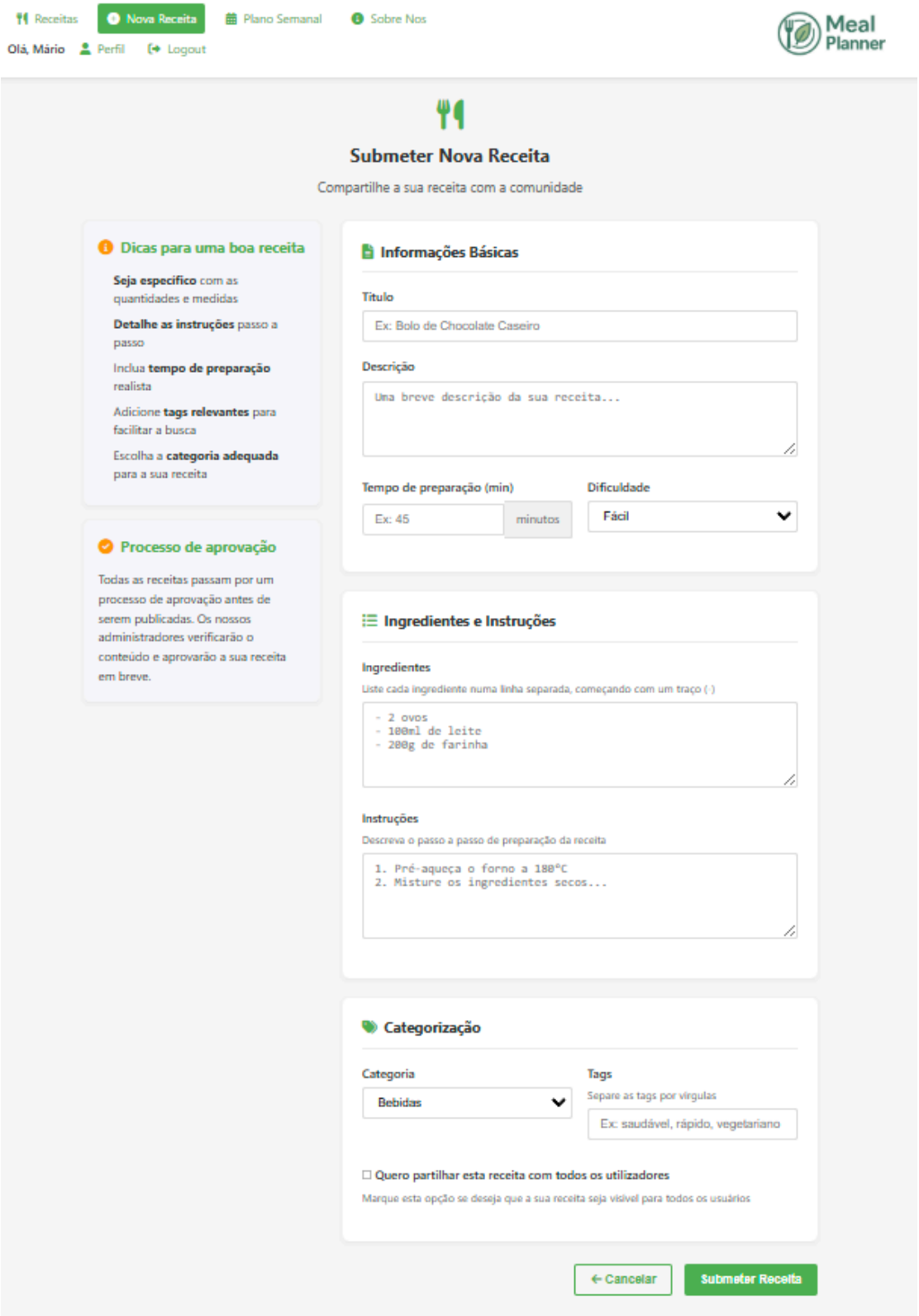


Figura 3.2.9 – Gestão de Utilizadores (Admin)

Print da página onde o admin vê, edita, bloqueia/desbloqueia utilizadores.

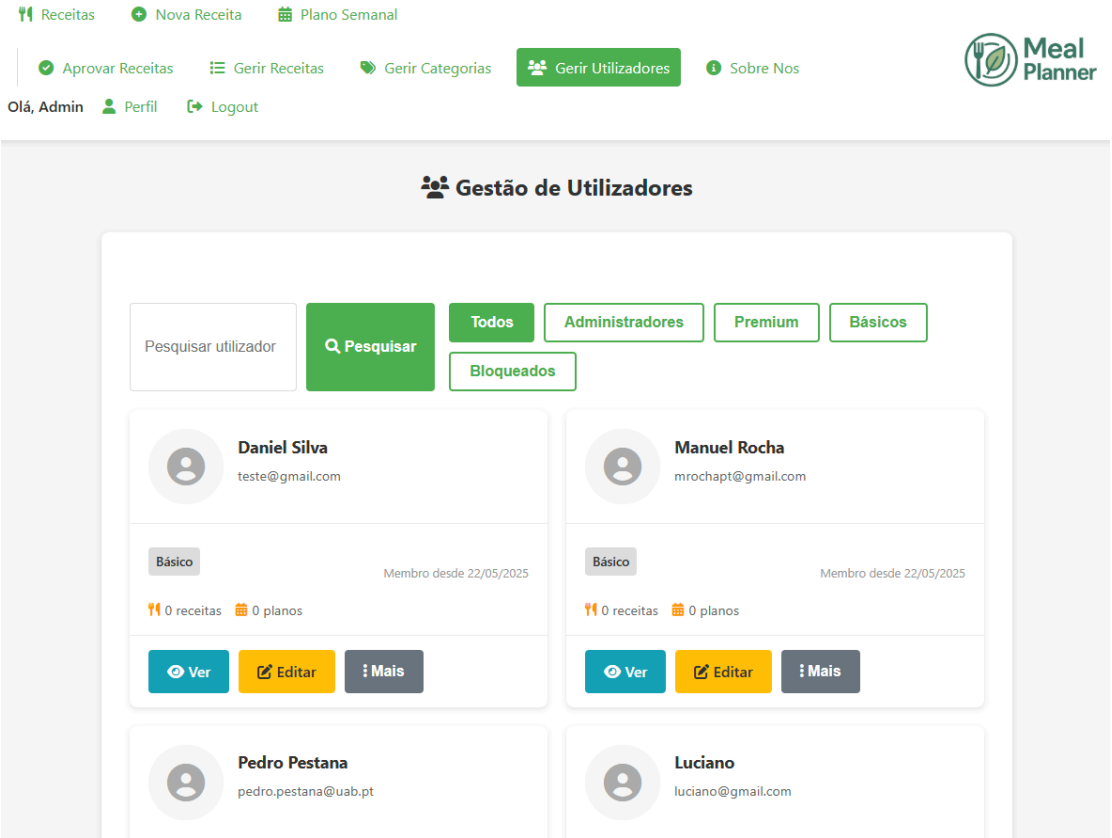


Figura 3.2.10 – Gestão de Categorias(Admin)

Print da página onde o admin cria, edita ou elimina categorias.

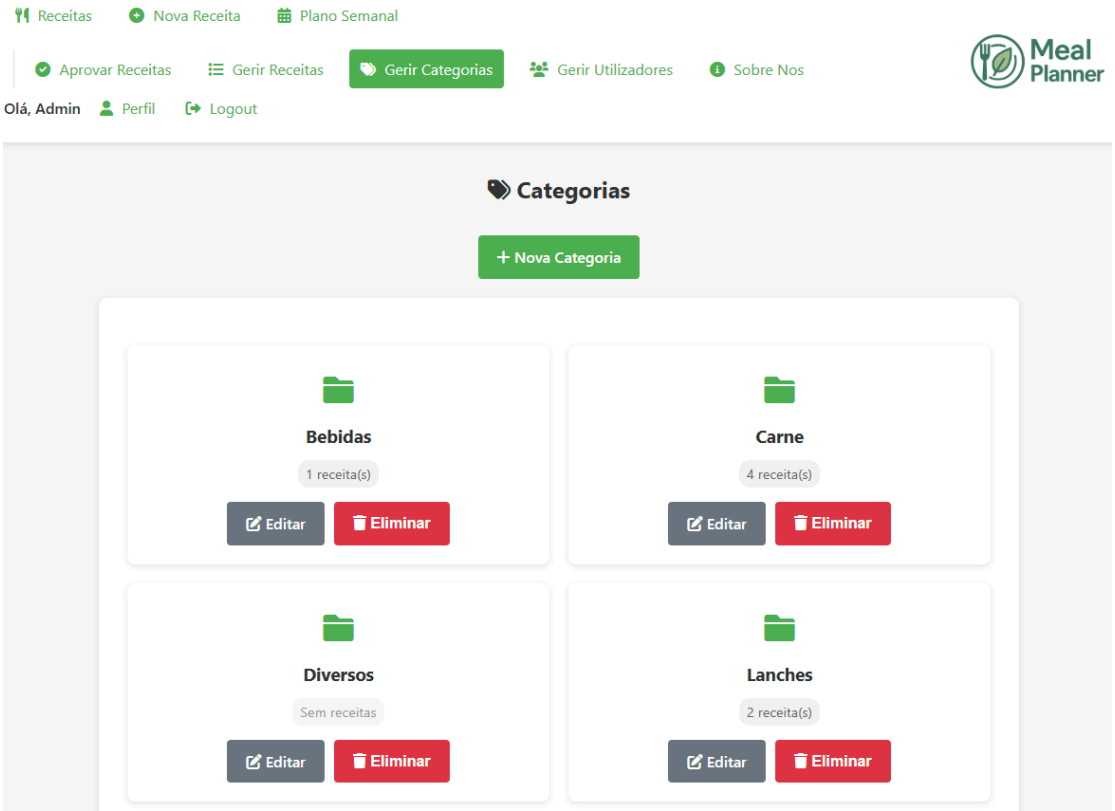


Figura 3.2.11 – Favoritos e Receitas Bloqueadas (Perfil do Utilizador)

Print da secção do perfil do utilizador onde vê as receitas favoritas e as receitas bloqueadas.

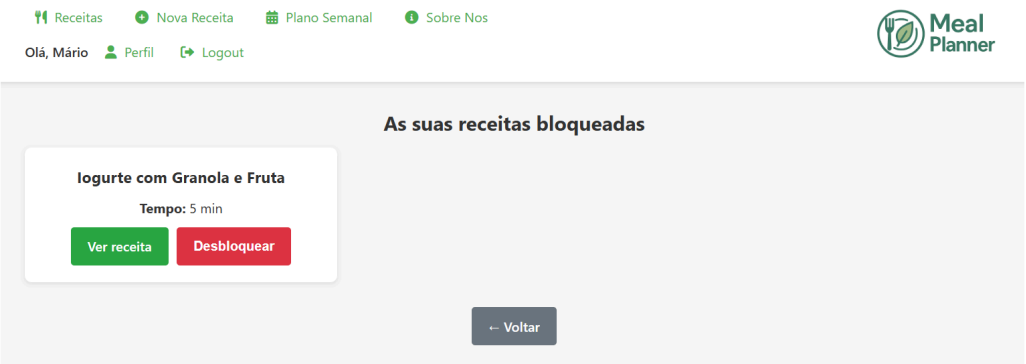


Figura 3.2.12 – Página de Aprovação de Receitas (Admin)

Print do painel onde o admin aprova/recusa receitas submetidas por outros utilizadores.

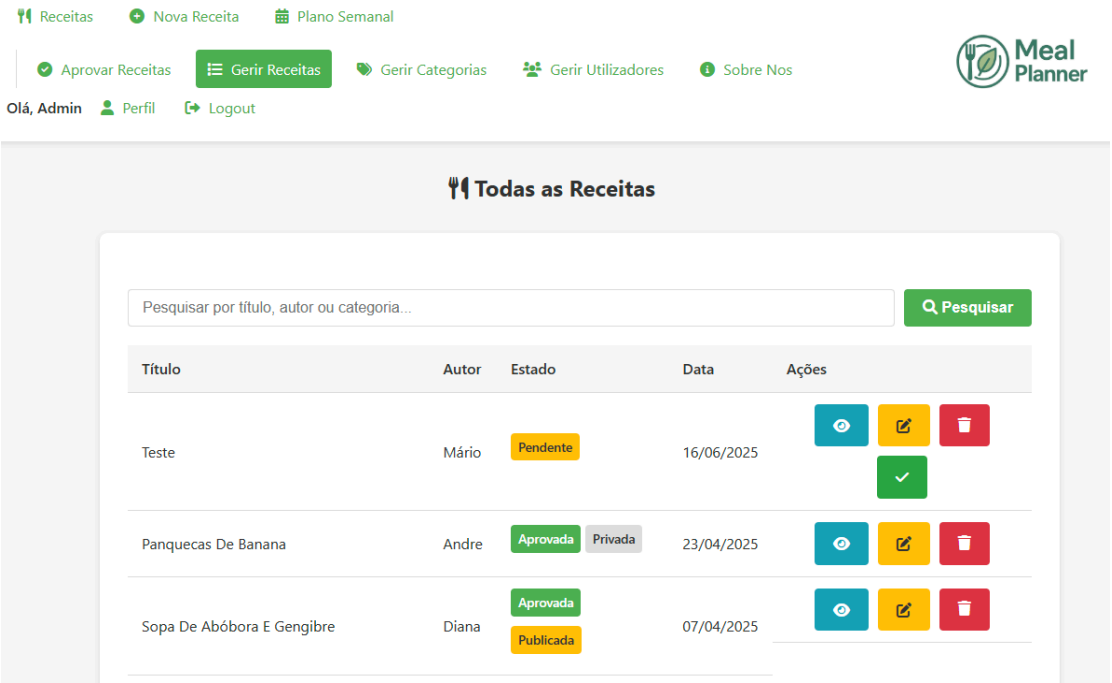


Figura 3.2.13 – Página de edição de plano semanal

Print do ecrã onde se edita ou elimina um plano semanal já criado.

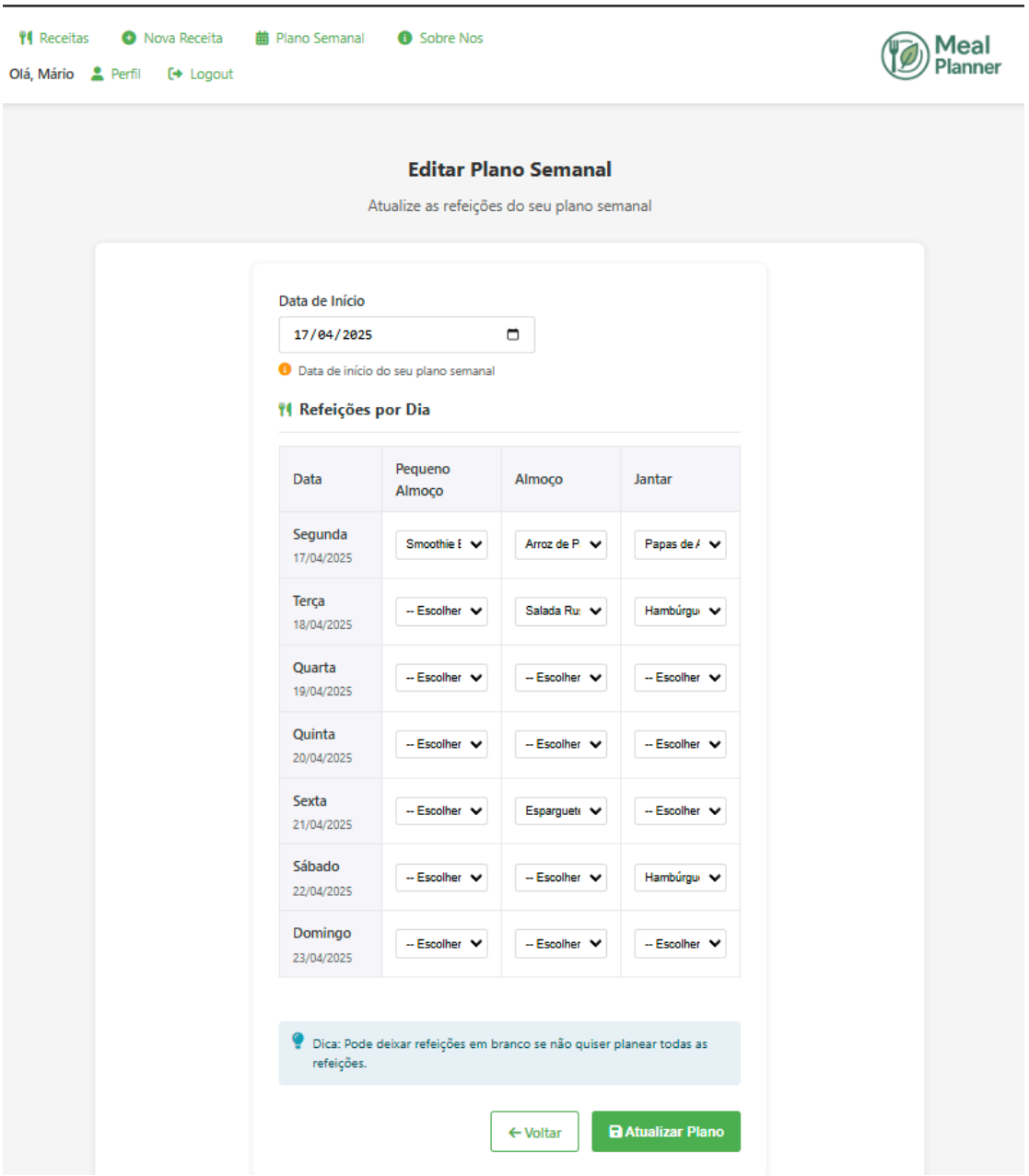


Figura 3.2.14 – Exportação de Plano Semanal em PDF

Exemplo de plano semanal exportado em formato PDF. **Esta funcionalidade encontra-se disponível quando a aplicação é executada localmente, uma vez que a plataforma Railway não suporta as bibliotecas necessárias para a geração de ficheiros PDF.**

Plano Semanal de Refeições

Data de início: 14/06/2025
Período: 14/06 - 20/06/2025

Resumo do Plano

Dias planejados: 7
Total de refeições: 21

Sábado, 14/06/2025

Refeição	Receita	Tempo
Pequeno almoço	Sopa De Abóbora E Gengibre Tags: inverno, vegetariano, sem glúten, saudável	45 min
Almoço	Salada De Quinoa Mediterrânea Tags: saudável, vegetariano, almoço, rápido	30 min
Jantar	Sangria de Frutos Vermelhos e Espumante Tags: bebida, festa, verão, rápida, refrescante	10 min

Domingo, 15/06/2025

Refeição	Receita	Tempo
Pequeno almoço	Panquecas com Fruta Tags: pequeno almoço, saudável, rápido	15 min
Almoço	Salada de Quinoa com Frango Tags: almoço, leve, proteína	25 min
Jantar	Sopa de Legumes Tags: jantar, leve, saudável	30 min

Segunda-feira, 16/06/2025

Refeição	Receita	Tempo
Pequeno almoço	Iogurte com Granola e Fruta Tags: pequeno almoço, rápido, leve	5 min
Almoço	Bacalhau à Brás Tags: almoço, tradicional, intenso	35 min
Jantar	Tortilha de Batata Tags: jantar, clássico, reconfortante	25 min

3.3 Manual de utilização

3.3.1 Registo e Autenticação

O utilizador deve criar uma conta através do formulário de registo, preenchendo nome, email e password. Após o registo, pode iniciar a sessão introduzindo as credenciais definidas. O logout pode ser efetuado a qualquer momento através do menu de navegação.

3.3.2 Pesquisa e Exploração de Receitas

Após o login, o utilizador pode aceder à secção de receitas, pesquisar por nome ou aplicar filtros como categoria alimentar e tempo de preparação. As receitas são apresentadas com detalhes relevantes, podendo ser consultadas individualmente.

3.3.3 Visualização de Receita

Ao selecionar uma receita, são apresentados todos os detalhes: descrição, ingredientes, instruções, categoria, tempo de preparação, dificuldade e tags. É possível adicionar receitas aos favoritos ou bloquear receitas para que deixem de ser sugeridas.

3.3.4 Submissão de Receitas

Os utilizadores podem submeter novas receitas, preenchendo o formulário com título, descrição, tempo de preparação, dificuldade, ingredientes, instruções, categoria e tags. É possível optar por tornar a receita pública ou privada. As receitas submetidas aguardam aprovação do administrador antes de ficarem visíveis publicamente.

3.3.5 Gestão de Favoritos e Receitas Bloqueadas

Através do perfil, o utilizador pode aceder à lista de receitas favoritas e remover receitas dessa lista. Pode ainda gerir receitas bloqueadas, desbloqueando-as caso queira voltar a vê-las nas sugestões.
As receitas bloqueadas não aparecem nas pesquisas nem na seleção de receitas ao criar ou editar um plano semanal, garantindo uma experiência personalizada e filtrada de acordo com as preferências do utilizador.

3.3.6 Criação e Gestão de Planos Semanais

Apenas as receitas não bloqueadas pelo utilizador estão disponíveis para seleção no momento da criação ou edição de um plano semanal.

O utilizador pode criar um plano semanal, selecionando receitas para pequeno-almoço, almoço e jantar de cada dia da semana. Os planos podem ser visualizados, editados, eliminados e exportados em PDF (apenas disponível localmente devido a limitações da plataforma Railway).

3.3.7 Funcionalidades de Administração

Utilizadores com perfil de administrador podem aceder ao painel de administração, onde podem:

- Aprovar ou recusar receitas submetidas por outros utilizadores;
- Gerir categorias (criar, editar, eliminar);
- Gerir utilizadores (bloquear/desbloquear contas, alterar nível de acesso, etc.);

Capítulo 4 - Resultados Obtidos

Foi possível implementar um conjunto robusto de funcionalidades essenciais na aplicação Meal Planner. A aplicação encontra-se estável nas suas principais operações, permitindo a interação fluida entre o utilizador, a lógica da aplicação e a base de dados.

4.1 Funcionalidades Ativas

As seguintes funcionalidades encontram-se implementadas e operacionais:

4.1.1 Tabela de funcionalidades

Funcionalidade	Estado
Registo e login de utilizadores	✓ Implementado
Logout e gestão de sessões	✓ Implementado
Criação e edição de plano semanal	✓ Implementado
Visualização de planos existentes	✓ Implementado
Pesquisa de receitas com filtros	✓ Implementado
Submissão de receitas públicas e privadas	✓ Implementado
Aprovação e recusa de receitas (admin)	✓ Implementado
Gestão de categorias (criar, editar, eliminar)	✓ Implementado
Gestão de utilizadores (bloquear, editar, eliminar)	✓ Implementado
Distinção de níveis (comum, premium, admin)	✓ Implementado

Funcionalidade de favoritos ou ocultar receitas	✓ Implementado
Exportação de planos	✓ Implementado (exportação para PDF disponível apenas localmente; a plataforma Railway não suporta esta funcionalidade online)

4.2 Integração entre Componentes

A aplicação foi desenvolvida com uma arquitetura full-stack, integrando frontend (HTML, CSS e Jinja2) e backend (Flask em Python), permitindo a comunicação entre as várias camadas através de rotas seguras e bem definidas.

A base de dados relacional encontra-se alojada na plataforma Railway (PostgreSQL) e comunica corretamente com a aplicação, permitindo operações de criação, consulta, atualização e eliminação de dados em tempo real.

4.3 Dados Atuais na Base de Dados

A base de dados da aplicação já contém a seguinte informação real:

- 9 utilizadores + 1 admin

- 5 planos semanais

- 26 receitas

- 10 categorias

Estes dados, alguns inseridos manualmente outros por terem sido convidadas algumas pessoas a testar o site **Meal Planner**, servem de base de testes e validação para as funcionalidades principais da aplicação.

4.4 Validação Manual e automático das Funcionalidades

Durante o desenvolvimento foram realizados **testes manuais** e **testes automáticos,** realçada a sua importância ao longo da unidade curricular Laboratório de Desenvolvimento de Software, para garantir o correto funcionamento de todas as funcionalidades implementadas.

Testes Manuais

Foram realizados testes manuais para validar o comportamento esperado das funcionalidades implementadas. Entre os testes realizados, destacam-se:

- Criação de contas e autenticação com diferentes perfis;
- Submissão de receitas públicas e privadas, com validação posterior no painel do administrador;
- Aplicação de filtros de pesquisa (por categoria e tempo), com retorno de resultados corretos;
- Criação e visualização de planos semanais com associação correta de receitas por dia/refeição;
- Acesso ao painel administrativo e execução das operações de gestão de utilizadores e categorias.

Os resultados destes testes foram positivos, não tendo sido detectadas falhas críticas.

Testes Automáticos

Adicionalmente, foi desenvolvido um conjunto de **31 testes automáticos** utilizando a framework pytest para Python. Estes testes abrangeram as principais áreas da aplicação, nomeadamente:

- Autenticação e gestão de sessões (login, logout, registo, credenciais inválidas, duplicação de contas);
- Proteção de rotas e permissões de acesso conforme o nível de utilizador (comum, admin, anónimo);
- Submissão, aprovação, recusa e bloqueio de receitas;
- Gestão de favoritos e receitas bloqueadas;
- Criação, edição e eliminação de planos semanais;
- Validação de redirecionamentos corretos para utilizadores não autenticados.

Todos os testes automáticos foram concluídos com sucesso, conforme apresentado na figura abaixo, não se tendo detectado falhas ou comportamentos inesperados nas funcionalidades testadas.

Figura 4.4.1 Resultado dos testes automáticos

```
========================================================================== test session starts ==========================================================================
platform win32 -- Python 3.12.8, pytest-8.3.5, pluggy-1.6.0 -- C:\Users\Mario Prazeres\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: E:\GIT Meal Planner\Meal-Planner
collected 31 items

app/tests/test_apagar_plano.py::test_apagar_plano_valido PASSED                                                                                                    [  3%]
app/tests/test_aprovar_receita_admin.py::test_aprovar_receita PASSED                                                                                               [  6%]
app/tests/test_auth_login.py::test_login_credenciais_validas PASSED                                                                                                 [  9%]
app/tests/test_auth_login_invalid.py::test_login_credenciais_invalidas PASSED                                                                                       [ 12%]
app/tests/test_bloquear_receita.py::test_bloquear_receita PASSED                                                                                                    [ 16%]
app/tests/test_bloquear_sem_login.py::test_bloquear_receita_sem_login PASSED                                                                                        [ 19%]
app/tests/test_criar_plano.py::test_criar_plano_valido PASSED                                                                                                       [ 22%]
app/tests/test_criar_plano_sem_login.py::test_acesso_sem_login_redireciona_para_login PASSED                                                                        [ 25%]
app/tests/test_editar_plano.py::test_editar_plano_valido PASSED                                                                                                     [ 29%]
app/tests/test_favoritar_receita.py::test_adicionar_remover_favorito PASSED                                                                                         [ 32%]
app/tests/test_favoritar_sem_login.py::test_adicionar_remover_ver_favorito_sem_login PASSED                                                                         [ 35%]
app/tests/test_homepage.py::test_home_page PASSED                                                                                                                    [ 38%]
app/tests/test_logout.py::test_logout PASSED                                                                                                                         [ 41%]
app/tests/test_protected_routes.py::test_acesso_a_criar_plano_sem_login PASSED                                                                                      [ 45%]
app/tests/test_recusar_receita_admin.py::test_recusar_receita PASSED                                                                                                 [ 48%]
app/tests/test_registo_utilizador.py::test_registo_valido PASSED                                                                                                     [ 51%]
app/tests/test_registo_utilizador.py::test_registo_email_duplicado PASSED                                                                                            [ 54%]
app/tests/test_rotas_admin.py::test_acesso_admin_bloqueado_para_utilizador_comum[/admin/utilizadores] PASSED                                                         [ 58%]
app/tests/test_rotas_admin.py::test_acesso_admin_bloqueado_para_utilizador_comum[/admin/categorias] PASSED                                                           [ 61%]
app/tests/test_rotas_admin.py::test_acesso_admin_bloqueado_para_utilizador_comum[/admin/aprovar_receitas] PASSED                                                     [ 64%]
app/tests/test_rotas_admin.py::test_acesso_admin_bloqueado_para_utilizador_comum[/admin/receitas] PASSED                                                             [ 67%]
app/tests/test_rotas_protegidas_anonimo.py::test_rotas_anonimo_redireciona_para_login[/home] PASSED                                                                 [ 70%]
app/tests/test_rotas_protegidas_anonimo.py::test_rotas_anonimo_redireciona_para_login[/receitas/] PASSED                                                            [ 74%]
app/tests/test_rotas_protegidas_anonimo.py::test_rotas_anonimo_redireciona_para_login[/receitas/submeter] PASSED                                                    [ 77%]
app/tests/test_rotas_protegidas_anonimo.py::test_rotas_anonimo_redireciona_para_login[/favorites/ver] PASSED                                                        [ 80%]
app/tests/test_rotas_protegidas_anonimo.py::test_rotas_anonimo_redireciona_para_login[/blocked/bloqueadas] PASSED                                                   [ 83%]
app/tests/test_rotas_protegidas_anonimo.py::test_rotas_anonimo_redireciona_para_login[/categorias/] PASSED                                                          [ 87%]
app/tests/test_rotas_protegidas_anonimo.py::test_rotas_anonimo_redireciona_para_login[/planos/criar] PASSED                                                         [ 90%]
app/tests/test_rotas_protegidas_anonimo.py::test_rotas_anonimo_redireciona_para_login[/planos/meus] PASSED                                                          [ 93%]
app/tests/test_submeter_receita.py::test_submeter_receita_valida PASSED                                                                                              [ 96%]
app/tests/test_submeter_receita_sem_login.py::test_submeter_receita_sem_login PASSED                                                                                [100%]

========================================================================== 31 passed in 10.53s ==========================================================================
```

test_apagar_plano.py:
Testa se um utilizador autenticado consegue eliminar um plano semanal existente com sucesso.
test_aprovar_receita_admin.py:
Testa se um administrador consegue aprovar uma receita pendente através do painel de administração.
test_auth_login.py:
Testa se um utilizador consegue fazer login com credenciais válidas.
test_auth_login_invalid.py:
Testa se o sistema apresenta uma mensagem de erro quando o utilizador tenta fazer login com credenciais erradas.
test_bloquear_receita.py:
Testa se um utilizador consegue bloquear e desbloquear uma receita, verificando se essas ações são registadas corretamente na base de dados.
test_bloquear_sem_login.py:
Testa se o sistema impede utilizadores não autenticados de bloquear, desbloquear ou visualizar receitas bloqueadas, mostrando a mensagem adequada.
test_criar_plano.py:
Testa se um utilizador autenticado consegue criar um novo plano semanal com sucesso.
test_criar_plano_sem_login.py:
Garante que utilizadores não autenticados são redirecionados para o login ao tentarem aceder à criação de plano semanal.
test_editar_plano.py:
Testa se um utilizador autenticado consegue editar um plano semanal existente com sucesso.
test_favoritar_receita.py:

Testa se um utilizador autenticado consegue adicionar e remover uma receita dos seus favoritos, verificando também o registo na base de dados.

test_favoritar_sem_login.py:

Testa se o sistema impede utilizadores não autenticados de adicionar, remover ou ver receitas favoritas, mostrando sempre a mensagem adequada.

test_homepage.py:

Testa se a página principal do Meal Planner está acessível (responde com sucesso à rota "/").

test_logout.py:

Testa se o logout limpa a sessão do utilizador e apresenta a mensagem de término de sessão com redirecionamento correto.

test_protected_routes.py:

Testa se as rotas protegidas obrigam o utilizador a autenticar-se, redirecionando para o login quando necessário.

test_recusar_receita_admin.py:

Testa se um administrador consegue recusar (eliminar) uma receita pendente através do painel de administração, removendo-a da base de dados.

test_registo_utilizador.py:

Testa o registo de um novo utilizador (com dados válidos) e impede o registo caso o email já exista.

test_rotas_admin.py:

Testa se um utilizador comum (nível 1) é impedido de aceder às rotas de administração, apresentando mensagem de acesso restrito.

test_rotas_protegidas_anonimo.py:

Testa se utilizadores não autenticados são sempre redirecionados para a página de login ao tentarem aceder a rotas protegidas.

test_submeter_receita.py:

Testa se um utilizador autenticado consegue submeter uma nova receita válida com sucesso.

test_submeter_receita_sem_login.py:

Garante que a página de submissão de receitas só está acessível a utilizadores autenticados, redirecionando para o login caso contrário.

Capítulo 5 - Conclusão

O desenvolvimento do **Meal Planner** constituiu um desafio exigente e enriquecedor, que permitiu consolidar e aplicar de forma prática os conhecimentos adquiridos ao longo da Licenciatura em Engenharia Informática. A implementação desta aplicação web, desde a modelação da base de dados até à interface do utilizador, exigiu a integração de várias áreas do curso, nomeadamente programação, bases de dados, engenharia de software, segurança informática e usabilidade.

Entre os principais **desafios encontrados**, destacam-se:

- A definição de uma arquitetura modular, que garantisse a separação clara entre frontend, backend e persistência de dados, facilitando a escalabilidade e manutenção futura;

- A conceção de uma base de dados robusta e flexível, capaz de responder a requisitos em evolução e suportar diferentes tipos de utilizadores e receitas;

- A escolha criteriosa das tecnologias e ferramentas, tendo em conta as limitações e vantagens de cada uma, como a decisão de recorrer a uma base de dados própria após a análise das APIs disponíveis;

- A implementação segura da autenticação, recorrendo ao hashing das passwords e validação rigorosa dos dados, seguindo as melhores práticas recomendadas (Silberschatz et al., 2020; Tavani, 2023);

- O equilíbrio entre funcionalidade e simplicidade, garantindo que a aplicação é intuitiva para o utilizador final, sem descurar a complexidade necessária para suportar funcionalidades avançadas.

Ao longo do projeto, foi necessário tomar decisões importantes, muitas vezes tendo de pesar entre soluções ideais e as limitações impostas pelo contexto real: limitações técnicas da plataforma Railway, ausência de APIs externas adequadas, ou o tempo disponível para implementar novas funcionalidades. Cada uma destas decisões foi orientada pelo rigor académico, mas também pela procura de soluções pragmáticas e sustentáveis.

O percurso realizado permitiu perceber a importância de um planeamento detalhado, da documentação rigorosa e dos testes (manuais e automáticos) na construção de software fiável. O contacto com ferramentas modernas (Flask, SQLAlchemy, WTForms, Railway, Git/GitHub, pytest) revelou-se essencial para o sucesso do projeto e aproximou a experiência académica das reais exigências do mercado de trabalho.

Em síntese, o **Meal Planner** atingiu plenamente os objetivos propostos:

- Permite registo e autenticação segura de utilizadores;
- Garante a pesquisa, submissão e gestão de receitas;
- Disponibiliza planeamento semanal, gestão de favoritos/bloqueados e painel administrativo;
- Apresenta uma arquitetura limpa, escalável e facilmente adaptável a futuras evoluções.

Naturalmente, há espaço para melhorias e expansão, como a integração de imagens, a criação de uma página pública de demonstração, ou o alargamento dos testes automáticos. No entanto, o trabalho desenvolvido constitui uma base sólida, robusta e extensível, evidenciando a importância e aplicabilidade da formação académica recebida.

O projeto não só permitiu consolidar competências técnicas, mas também desenvolver capacidades transversais, como trabalho em equipa, resolução de problemas, espírito crítico e autonomia — competências essas que serão fundamentais para futuros desafios profissionais e académicos.

Considera-se que o Meal Planner, além de atingir todos os objetivos definidos, constitui uma base sólida para futuras evoluções e para a aplicação prática dos conhecimentos adquiridos em Engenharia Informática.

Gostaríamos ainda de expressar o nosso agradecimento ao professor Pedro Pestana, cujo acompanhamento, disponibilidade e sugestões ao longo do desenvolvimento deste projeto foram determinantes para a superação dos desafios encontrados e para a evolução do trabalho. O seu contributo revelou-se fundamental para a consolidação dos conhecimentos adquiridos e para a concretização de uma solução prática e robusta.

Bibliografia

Bagga, J. (2023). *Introduction to APIs*. In *Introduction to Integration Suite Capabilities* (pp. 1–8). Springer.

Burkett, W. C. (1997). Database schema design quality principles. *ResearchGate*. Recuperado em 22 de abril de 2025, de https://www.researchgate.net/publication/2420913

Davis, J. S., & Yen, J. (2002). Gantt charts: A centenary appreciation. *European Journal of Operational Research, 149*(2), 430–437.

Foster, J., et al. (2023). A qualitative study of REST API design and specification practices. *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*.

Flask Documentation. (2024). Recuperado em 16 de junho de 2025, de https://flask.palletsprojects.com/

Geraldi, J., & Lechler, T. (2012). Gantt charts revisited: A critical analysis of its roots and implications to the management of projects today. *International Journal of Managing Projects in Business, 5*(4), 578–594.

Manikas, K. (2016). Application programming interfaces (APIs) form an integral component of the software ecosystem. *International Journal of Enterprise Information Systems, 15*(3), 76–93.

Peinl, P. (2020). Teaching database design and the importance of integrity.

pytest Documentation. (2024). Recuperado em 16 de junho de 2025, de https://docs.pytest.org/en/stable/

Railway Documentation. (2024). Recuperado em 16 de junho de 2025, de https://docs.railway.app/

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill Education.

SQLAlchemy Documentation. (2024). Recuperado em 16 de junho de 2025, de https://docs.sqlalchemy.org/

Tavani, H. T. (2023). *Ethics and technology* (6th ed.). Wiley.

Werkzeug Documentation. (2024). Recuperado em 16 de junho de 2025, de https://werkzeug.palletsprojects.com/en/latest/

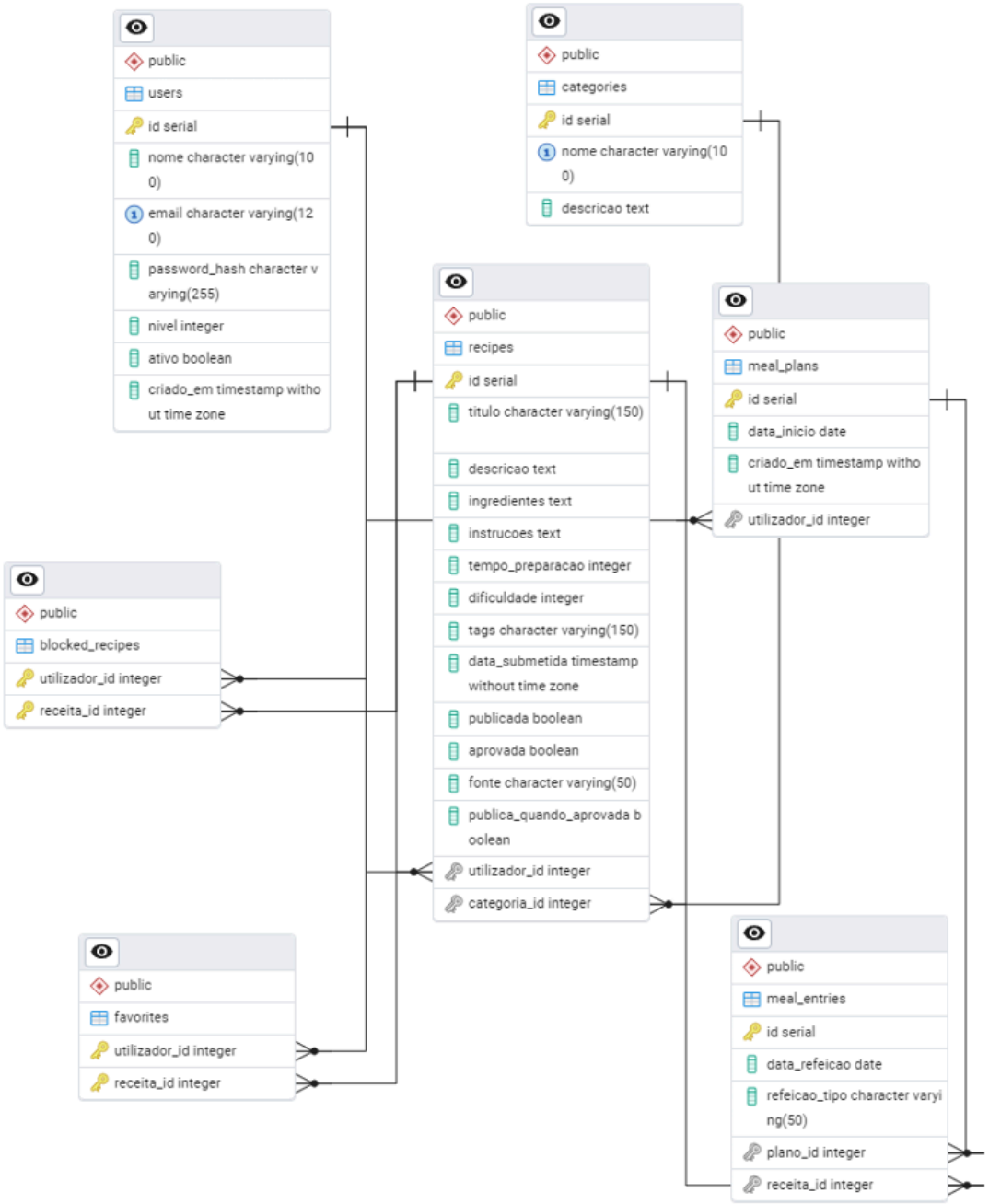
WTForms Documentation. (2024). Recuperado em 16 de junho de 2025, de https://wtforms.readthedocs.io/

Anexos

Anexo I - Diagrama da Base de Dados

Figura A1: Diagrama relacional da base de dados do Meal Planner

Diagrama ER que representa todas as entidades e relações da base de dados, incluindo utilizadores, receitas, planos semanais, categorias, favoritos e receitas bloqueadas.



O diagrama acima (Figura A1) representa o modelo relacional completo da base de dados do **Meal Planner**, com destaque para as entidades principais, respectivos

atributos e as relações entre tabelas (incluindo favoritos e receitas bloqueadas). As chaves primárias e estrangeiras estão devidamente identificadas, garantindo a integridade referencial do sistema..

Anexo II – Código SQL de Criação das Tabelas

Este anexo apresenta o código SQL utilizado para criar as principais tabelas da base de dados relacional do projeto Meal Planner. As instruções foram definidas com base na estrutura desenhada durante a fase de planeamento e encontram-se atualmente implementadas na base de dados PostgreSQL alojada na Railway.

Tabela: users

```
CREATE TABLE users (
        id SERIAL PRIMARY KEY,
        nome VARCHAR(100) NOT NULL,
        email VARCHAR(120) NOT NULL UNIQUE,
        password_hash VARCHAR(255) NOT NULL,
        nivel INTEGER,
        criado_em TIMESTAMP,
        ativo BOOLEAN DEFAULT TRUE
);
```

Tabela: categories

```
CREATE TABLE categories (
        id SERIAL PRIMARY KEY,
        nome VARCHAR(100) NOT NULL UNIQUE,
        descricao TEXT
);
```

Tabela: recipes

```
CREATE TABLE recipes (
        id SERIAL PRIMARY KEY,
        titulo VARCHAR(150) NOT NULL,
        descricao TEXT NOT NULL,
        ingredientes TEXT,
        instrucoes TEXT NOT NULL,
        tempo_preparacao INTEGER NOT NULL,
        dificuldade INTEGER NOT NULL,
        tags VARCHAR,
        data_submetida TIMESTAMP,
        publicada BOOLEAN,
        fonte VARCHAR,
```

```
        utilizador_id INTEGER NOT NULL,
        categoria_id INTEGER NOT NULL,
        aprovada BOOLEAN DEFAULT FALSE,
        publica_quando_aprovada BOOLEAN DEFAULT FALSE,
        FOREIGN KEY (utilizador_id) REFERENCES users(id),
        FOREIGN KEY (categoria_id) REFERENCES categories(id)
);
```

Tabela: meal_plans

```
CREATE TABLE meal_plans (
        id SERIAL PRIMARY KEY,
        data_inicio DATE NOT NULL,
        criado_em TIMESTAMP,
        utilizador_id INTEGER NOT NULL,
        FOREIGN KEY (utilizador_id) REFERENCES users(id)
);
```

Tabela: meal_entries

```
CREATE TABLE meal_entries (
        id SERIAL PRIMARY KEY,
        data_refeicao DATE NOT NULL,
        refeicao_tipo VARCHAR(50) NOT NULL,
        plano_id INTEGER NOT NULL,
        receita_id INTEGER NOT NULL,
        FOREIGN KEY (plano_id) REFERENCES meal_plans(id),
        FOREIGN KEY (receita_id) REFERENCES recipes(id)
);
```

Tabela: favorites

```
CREATE TABLE favorites (
   utilizador_id INTEGER NOT NULL,
   receita_id INTEGER NOT NULL,
   PRIMARY KEY (utilizador_id, receita_id),
   FOREIGN KEY (utilizador_id) REFERENCES users(id),
   FOREIGN KEY (receita_id) REFERENCES recipes(id)
);
```

Tabela: blocked_recipes

```
CREATE TABLE blocked_recipes (
   utilizador_id INTEGER NOT NULL,
   receita_id INTEGER NOT NULL,
   PRIMARY KEY (utilizador_id, receita_id),
   FOREIGN KEY (utilizador_id) REFERENCES users(id),
   FOREIGN KEY (receita_id) REFERENCES recipes(id)
);
```

O código acima garante a integridade referencial entre tabelas e reflete o modelo de dados implementado e apresentado no diagrama relacional do Anexo I.

Anexo III – Estrutura de Diretórios e Ficheiros do Projeto Meal Planner

Este anexo apresenta a estrutura de pastas e ficheiros do projeto Meal Planner, desenvolvido com o framework Flask. A aplicação encontra-se organizada de forma modular, separando as responsabilidades por diretórios específicos: rotas, modelos, formulários, serviços, templates, ficheiros estáticos, entre outros.

Estrutura de diretórios:

```
Meal-Planner/
├── app/
│   ├── __init__.py
│   ├── db.py
│   ├── forms/
│   │   ├── auxiliares.py
│   │   ├── category_form.py
│   │   ├── login_form.py
│   │   ├── meal_plan_form.py
│   │   ├── recipe_form.py
│   │   ├── register_form.py
│   │   └── user_admin_form.py
│   │   └── favorites_form.py
│   │   └── unblock_form.py
│   │   └── remove_favorites_form.py
│   │   └── block_recipe_form.py
│   ├── models/
│   │   ├── category.py
│   │   ├── meal_plan.py
│   │   ├── recipe.py
│   │   └── user.py
```

```
│   │   └── __init__.py
│   │   └── blocked_recipe.py
│   │   └── favorites.py
│   ├── routes/
│   │   ├── admin.py
│   │   ├── auth.py
│   │   ├── categories.py
│   │   ├── main.py
│   │   ├── meal_plans.py
│   │   └── recipes.py
│   │   └── blocked.py
│   │   └── favorites.py
│   ├── services/
│   │   ├── category_service.py
│   │   ├── meal_plan_service.py
│   │   ├── recipe_service.py
│   │   └── user_service.py
│   │   └── blocked_service.py
│   │   └── favorites_service.py
│   ├── static/
│   │   └── css/
│   │   └── style.css
│   │   └── meal_planner_transparent_logo.png
│   ├── templates/
│   │   ├── base.html
│   │   ├── base_pdf.html
│   │   ├── admin/
│   │   │   ├── aprovar_receitas.html
│   │   │   ├── categorias.html
│   │   │   ├── criar_categoria.html
│   │   │   ├── editar_categoria.html
│   │   │   ├── editar_receita.html
│   │   │   ├── editar_utilizador.html
│   │   │   ├── listar_receitas.html
│   │   │   ├── painel.html
│   │   │   ├── receitas_pendentes.html
│   │   │   ├── utilizadores.html
│   │   │   └── ver_utilizador.html
│   │   ├── auth/
│   │   │   ├── login.html
│   │   │   └── register.html
│   │   ├── categories/
│   │   │   ├── index.html
│   │   │   └── ver.html
│   │   ├── main/
│   │   │   └── home.html
```

```
│   │   │   └── start.html
│   │   │   └── about.html
│   │   ├── meal_plans/
│   │   │   ├── criar.html
│   │   │   ├── editar.html
│   │   │   ├── meus.html
│   │   │   ├── ver.html
│   │   │   └── ver_pdf.html
│   │   └── recipes/
│   │   ├── listar.html
│   │   ├── submeter.html
│   │   └── bloqueadas.html
│   │   └── favorites.html
│   │   └── ver_bloqueadas.html
│   │   └── ver_favoritas.html
│   │   └── ver.html
│── instance/
│     └─ mealplanner.db
├── .gitignore
├── Procfile
├── config.py
├── criar_admin.py
├── migrar_todas_as_tabelas.py
├── requirements.txt
├── run.py
├── seed_oficiais.py
└── setup_db.py
```

A pasta `app/` é o núcleo da aplicação Flask. Contém a inicialização da app (`__init__.py`), a ligação à base de dados (`db.py`), os ficheiros de rotas, modelos, serviços, formulários, templates e ficheiros estáticos. Os templates estão organizados por secções temáticas como `admin`, `auth`, `meal_plans` e `recipes`, garantindo manutenção e escalabilidade mais fáceis.

Anexo IV – Exemplo de Plano Semanal

Este anexo apresenta os principais componentes técnicos envolvidos na funcionalidade de criação de um plano semanal na aplicação Meal Planner. Abaixo estão incluídos o modelo de dados, a rota Flask responsável pela lógica de criação, o formulário WTForms utilizado e o template HTML correspondente.

1. Modelo SQLAlchemy (models/meal_plan.py)

```
class MealPlan(db.Model):
        __tablename__ = "meal_plans"
```

```
id = db.Column(db.Integer, primary_key=True, autoincrement=True)
data_inicio = db.Column(db.Date, nullable=False)
criado_em = db.Column(db.DateTime, default=datetime.utcnow)
utilizador_id = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=False)
utilizador = db.relationship("User", backref=db.backref("planos", lazy=True))

class MealEntry(db.Model):
    __tablename__ = "meal_entries"
    id = db.Column(db.Integer, primary_key=True)
    data_refeicao = db.Column(db.Date, nullable=False)
    refeicao_tipo = db.Column(db.String(50), nullable=False)
    plano_id = db.Column(db.Integer, db.ForeignKey("meal_plans.id"), nullable=False)
    plano = db.relationship("MealPlan", backref=db.backref("refeicoes", cascade="all, delete-orphan", lazy=True))
    receita_id = db.Column(db.Integer, db.ForeignKey("recipes.id"), nullable=False)
    receita = db.relationship("Recipe")
```

2. Formulário WTForms (forms/meal_plan_form.py)

```
class MealPlanForm(FlaskForm):
    data_inicio = DateField("Data de Início", validators=[DataRequired()])
    submit = SubmitField("Criar Plano")
```

3. Rota Flask (routes/meal_plans.py)

```
@meal_plans_bp.route("/criar", methods=["GET", "POST"])
def criar():
    if "user_id" not in session:
    flash("Tens de iniciar sessão para criar um plano semanal.", "warning")
    return redirect(url_for("auth.login"))

    form = MealPlanForm()
    receitas = listar_receitas()

    if form.validate_on_submit():
    dias = ["segunda", "terça", "quarta", "quinta", "sexta", "sábado", "domingo"]
    refeicoes = ["pequeno_almoço", "almoço", "jantar"]
    refeicoes_selecionadas = {}

     for dia in dias:
    for refeicao in refeicoes:
    chave = f"{dia}_{refeicao}"
    receita_id = request.form.get(chave)
    if receita_id:
```

```
            index = dias.index(dia)
            data_refeicao = form.data_inicio.data + timedelta(days=index)
        refeicoes_selecionadas[(data_refeicao, refeicao)] = int(receita_id)

    plano = criar_plano_semanal(
    utilizador_id=session["user_id"],
    data_inicio=form.data_inicio.data,
    refeicoes_selecionadas=refeicoes_selecionadas,
    )

    flash("Plano criado com sucesso!", "success")
    return redirect(url_for("main.home"))

    return render_template("meal_plans/criar.html", form=form, receitas=receitas)
```

4. Template HTML (templates/meal_plans/criar.html)

```
<form method="POST" class="form">
  {{ form.csrf_token }}
  {{ form.data_inicio(class="input-field date-picker") }}
  {% for dia in dias %}
  <tr>
        <td>{{ dia.title() }}</td>
        {% for refeicao in refeicoes %}
        <td>
        <select name="{{ dia }}_{{ refeicao }}">
        <option value="">-- Escolher receita --</option>
        {% for r in receitas %}
        <option value="{{ r.id }}">{{ r.titulo }}</option>
        {% endfor %}
        </select>
        </td>
        {% endfor %}
  </tr>
  {% endfor %}
  {{ form.submit(class="btn btn-primary") }}
 </form>
```

Este fluxo permite ao utilizador construir, de forma personalizada, o seu plano alimentar semanal, escolhendo receitas distintas para cada dia e refeição, com todos os dados validados e persistidos na base de dados.

Anexo V – Modelos SQLAlchemy

Este anexo apresenta os modelos de dados definidos com SQLAlchemy no projeto Meal Planner. Estes modelos representam as tabelas da base de dados e definem os seus atributos, relacionamentos e comportamentos auxiliares.

User (models/user.py)

```
class User(db.Model):
        __tablename__ = "users"

        id = db.Column(db.Integer, primary_key=True, autoincrement=True)
        nome = db.Column(db.String(100), nullable=False)
        email = db.Column(db.String(120), unique=True, nullable=False)
        password_hash = db.Column(db.String(255), nullable=False)
        nivel = db.Column(db.Integer, default=1)  # 1: comum, 2: premium, 3: admin
        ativo = db.Column(db.Boolean, default=True)
        criado_em = db.Column(db.DateTime, default=datetime.utcnow)

        def set_password(self, password):
        self.password_hash = generate_password_hash(password)

        def check_password(self, password):
        return check_password_hash(self.password_hash, password)

        def is_admin(self):
        return self.nivel == 3

        def is_premium(self):
        return self.nivel == 2

        def to_dict(self):
        return {
        "id": self.id,
        "nome": self.nome,
        "email": self.email,
        "nivel": self.nivel,
        "ativo": self.ativo,
        "criado_em": self.criado_em.isoformat(),
        }

        def __repr__(self):
        return f"<User {self.email} (Nível: {self.nivel}, Ativo: {self.ativo})>"
```

Recipe (models/recipe.py)

```
class Recipe(db.Model):
        __tablename__ = "recipes"
```

```
    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    titulo = db.Column(db.String(150), nullable=False)
    descricao = db.Column(db.Text, nullable=False)
    ingredientes = db.Column(db.Text, nullable=False)
    instrucoes = db.Column(db.Text, nullable=False)
    tempo_preparacao = db.Column(db.Integer, nullable=False)
    dificuldade = db.Column(db.Integer, nullable=False)
    tags = db.Column(db.String(150))
    data_submetida = db.Column(db.DateTime, default=datetime.utcnow)
    publicada = db.Column(db.Boolean, default=False)
    aprovada = db.Column(db.Boolean, default=False)
    fonte = db.Column(db.String(50), default="utilizador")
    publica_quando_aprovada = db.Column(db.Boolean, default=False)

    utilizador_id = db.Column(db.Integer, db.ForeignKey("users.id"),
nullable=False)
    autor = db.relationship("User", backref=db.backref("receitas", lazy=True))

    categoria_id = db.Column(db.Integer, db.ForeignKey("categories.id"),
nullable=False)
    categoria = db.relationship("Category", backref=db.backref("receitas",
lazy=True))

    def __repr__(self):
    return f"<Recipe {self.titulo}>"
```

MealPlan & MealEntry (models/meal_plan.py)

```
class MealPlan(db.Model):
    __tablename__ = "meal_plans"

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    data_inicio = db.Column(db.Date, nullable=False)
    criado_em = db.Column(db.DateTime, default=datetime.utcnow)
    utilizador_id = db.Column(db.Integer, db.ForeignKey("users.id"),
nullable=False)
    utilizador = db.relationship("User", backref=db.backref("planos", lazy=True))

 class MealEntry(db.Model):
    __tablename__ = "meal_entries"

    id = db.Column(db.Integer, primary_key=True)
    data_refeicao = db.Column(db.Date, nullable=False)
    refeicao_tipo = db.Column(db.String(50), nullable=False)
    plano_id = db.Column(db.Integer, db.ForeignKey("meal_plans.id"),
```

```
nullable=False)
        plano = db.relationship("MealPlan", backref=db.backref("refeicoes",
cascade="all, delete-orphan", lazy=True))
        receita_id = db.Column(db.Integer, db.ForeignKey("recipes.id"), nullable=False)
        receita = db.relationship("Recipe")
```

Category (models/category.py)

```
class Category(db.Model):
        __tablename__ = "categories"

        id = db.Column(db.Integer, primary_key=True, autoincrement=True)
        nome = db.Column(db.String(100), unique=True, nullable=False)
        descricao = db.Column(db.Text)

        def __repr__(self):
        return f"<Categoria {self.nome}>"
```

Favorites (models/favorites.py)

```
class Favorites(db.Model):
    __tablename__ = "favorites"

    utilizador_id = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=False)
    receita_id = db.Column(db.Integer, db.ForeignKey("recipes.id"), nullable=False)

    # Impede que um utilizador adicione a mesma receita várias vezes

    __table_args__ = (

        db.PrimaryKeyConstraint("utilizador_id", "receita_id"),

    )

    def __repr__(self):

        return f"<User:{self.utilizador_id} Recipe id:{self.receita_id}>"

    def to_dict(self):

        return {

            "id_receita": self.receita_id,

            "id_utilizador": self.utilizador_id,

        }
```

BlockedRecipe (models/blocked_recipe.py)

```
class BlockedRecipe(db.Model):
    __tablename__ = "blocked_recipes"
    utilizador_id = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=False)
    receita_id = db.Column(db.Integer, db.ForeignKey("recipes.id"), nullable=False)
    __table_args__ = (
        db.PrimaryKeyConstraint("utilizador_id", "receita_id"),
    )
    def __repr__(self):
        return f"<BlockedRecipe user:{self.utilizador_id} receita:{self.receita_id}>"
```

Anexo VI – Validação com WTForms

Este anexo apresenta exemplos dos formulários implementados no projeto Meal Planner com recurso à biblioteca WTForms. Os formulários são utilizados para validar a entrada de dados do utilizador no lado do servidor, garantindo maior robustez e segurança da aplicação.

1. Formulário de Registo (forms/register_form.py)

```
class RegisterForm(FlaskForm):
    nome = StringField("Nome", validators=[DataRequired(), Length(min=2, max=100)])
    email = StringField("Email", validators=[DataRequired(), Email()])
    password = PasswordField(
    "Password",
    validators=[
    DataRequired(),
    Length(min=6, message="A password deve ter pelo menos 6 caracteres."),
    ],
    )
    confirmar_password = PasswordField(
    "Confirmar Password",
    validators=[
    DataRequired(),
        EqualTo("password", message="As passwords não coincidem."),
    ],
    )
    submit = SubmitField("Criar Conta")
```

2. Formulário de Submissão de Receita (forms/recipe_form.py)

```
class RecipeForm(FlaskForm):
    titulo = StringField("Título", validators=[DataRequired(), Length(max=150)])
    descricao = TextAreaField("Descrição", validators=[DataRequired()])
    ingredientes = TextAreaField("Ingredientes", validators=[DataRequired()])
    instrucoes = TextAreaField("Instruções", validators=[DataRequired()])
    tempo_preparacao = IntegerField(
    "Tempo de preparação (min)", validators=[DataRequired(),
NumberRange(min=1)]
    )
    dificuldade = SelectField(
    "Dificuldade",
    choices=[(1, "Fácil"), (2, "Médio"), (3, "Difícil")],
    coerce=int,
    validators=[DataRequired()],
    )
    tags = StringField("Tags", validators=[Length(max=150)])
    categoria_id = SelectField("Categoria", coerce=int, validators=[DataRequired()])
    publica = BooleanField("Quero partilhar esta receita com todos os
utilizadores")
    submit = SubmitField("Submeter Receita")
```

3. Formulário de Login (forms/login_form.py)

```
class LoginForm(FlaskForm):
    email = StringField("Email", validators=[DataRequired(), Email()])
    password = PasswordField("Password", validators=[DataRequired()])
    submit = SubmitField("Entrar")
```

A utilização de WTForms permite validar todos os dados recebidos do utilizador antes do processamento pela aplicação, evitando submissões inválidas, ataques simples e facilitando a manutenção do código. Todos os formulários implementam regras de validação adequadas ao contexto da aplicação, promovendo a integridade e a segurança dos dados.

Anexo VII – Submissão de Receita

Este anexo descreve a funcionalidade de submissão de receitas na aplicação Meal Planner. Esta funcionalidade permite ao utilizador preencher um formulário com os detalhes da receita, que será submetida e colocada em estado pendente para aprovação por um administrador. Abaixo apresenta-se a rota Flask responsável pela lógica, bem como o template HTML correspondente.

1. Rota Flask (routes/recipes.py)

```
@recipes_bp.route("/submeter", methods=["GET", "POST"])
 def submeter():
        if "user_id" not in session:
        flash("Tens de iniciar sessão para submeter receitas.", "warning")
        return redirect(url_for("auth.login"))

        form = RecipeForm()
        categorias = listar_categorias()
        form.categoria_id.choices = [(c.id, c.nome) for c in categorias]

        if form.validate_on_submit():
        dados = {
        "titulo": form.titulo.data,
        "descricao": form.descricao.data,
        "ingredientes": form.ingredientes.data,
        "instrucoes": form.instrucoes.data,
        "tempo_preparacao": form.tempo_preparacao.data,
        "dificuldade": form.dificuldade.data,
        "tags": form.tags.data,
        "categoria_id": form.categoria_id.data,
        "publicada": False,
        "aprovada": True if session.get("user_nivel") == 3 else False,
        "fonte": "utilizador",
        }

        publica_quando_aprovada = form.publica.data

        nova_receita = criar_receita(dados, session["user_id"],
publica_quando_aprovada)
        flash("Receita submetida com sucesso! Aguarda aprovação.", "success")
        return redirect(url_for("recipes.listar"))

        return render_template("recipes/submeter.html", form=form)
```

2. Template HTML (templates/recipes/submeter.html)

O template HTML apresenta um formulário dividido por secções temáticas. Inclui instruções úteis para o utilizador e validações visuais para cada campo.

```
<form method="POST" class="recipe-form">
  {{ form.csrf_token }}

  <!-- Título -->
  {{ form.titulo.label }} {{ form.titulo(class="input-field") }}
```

```
  <!-- Descrição -->
  {{ form.descricao.label }} {{ form.descricao(class="input-field") }}

  <!-- Tempo e Dificuldade -->
  {{ form.tempo_preparacao.label }} {{ form.tempo_preparacao(class="input-field") }}
  {{ form.dificuldade.label }} {{ form.dificuldade(class="input-field") }}

  <!-- Ingredientes -->
  {{ form.ingredientes.label }} {{ form.ingredientes(class="input-field") }}

  <!-- Instruções -->
  {{ form.instrucoes.label }} {{ form.instrucoes(class="input-field") }}

  <!-- Categoria e Tags -->
  {{ form.categoria_id.label }} {{ form.categoria_id(class="input-field") }}
  {{ form.tags.label }} {{ form.tags(class="input-field") }}

  <!-- Checkbox para publicação após aprovação -->
  {{ form.publica() }} {{ form.publica.label }}

  <!-- Botões -->
  {{ form.submit(class="btn btn-primary") }}
 </form>
```

Após a submissão, a receita fica pendente de aprovação, exceto no caso de utilizadores com nível 3 (administradores), que podem aprovar automaticamente. A opção de marcar a receita como pública é guardada e aplicada apenas quando a aprovação for concluída com sucesso.

Anexo VIII – Autenticação

Este anexo descreve o mecanismo de autenticação de utilizadores implementado no projeto Meal Planner. A autenticação é realizada com base em formulários WTForms, hashing seguro de passwords, validação com SQLAlchemy e utilização de sessões Flask para manter o estado do utilizador autenticado.

1. Rota de Login (routes/auth.py)

```
@auth_bp.route("/login", methods=["GET", "POST"])
 def login():
        form = LoginForm()
        if form.validate_on_submit():
        utilizador = User.query.filter_by(email=form.email.data).first()
        if utilizador and utilizador.check_password(form.password.data):
```

```
        if not utilizador.ativo:
        flash("A tua conta encontra-se inativa.", "danger")
        return redirect(url_for("auth.login"))
        session["user_id"] = utilizador.id
        session["user_nome"] = utilizador.nome
        session["user_nivel"] = utilizador.nivel
        flash("Login efetuado com sucesso!", "success")
        return redirect(url_for("main.home"))
        else:
        flash("Credenciais inválidas. Tenta novamente.", "danger")
        return render_template("auth/login.html", form=form)
```

2. Rota de Logout (routes/auth.py)

```
@auth_bp.route("/logout")
 def logout():
        session.clear()
        flash("Sessão terminada com sucesso.", "info")
        return redirect(url_for("main.home"))
```

3. Métodos de Autenticação (models/user.py)

```
def set_password(self, password):
        self.password_hash = generate_password_hash(password)

 def check_password(self, password):
        return check_password_hash(self.password_hash, password)

 def is_admin(self):
        return self.nivel == 3

 def is_premium(self):
        return self.nivel == 2
```

4. Gestão de Sessão e Acesso

A aplicação utiliza o dicionário session do Flask para manter o estado do utilizador autenticado.

Após o login bem-sucedido, são guardadas as variáveis user_id, user_nome e user_nivel, que são utilizadas em todas as partes da aplicação para controlar o acesso a funcionalidades específicas.

- **Utilizadores com nível 3** têm permissões administrativas.
- **Utilizadores com nível 2** são considerados premium.

A proteção de rotas é feita manualmente através de verificações nas funções de rota, como:

```
if 'user_id' not in session:

    return redirect(url_for('auth.login'))
```

Nota adicional sobre segurança:

A utilização de hashing seguro para as passwords, através da biblioteca Werkzeug, garante que as palavras-passe nunca sejam armazenadas em texto simples na base de dados. Esta abordagem segue as melhores práticas de segurança recomendadas na literatura (Silberschatz et al., 2020; Tavani, 2023).

Anexo IX – Gestão de Utilizadores

Este anexo apresenta a funcionalidade de gestão de utilizadores na aplicação Meal Planner. Através do painel de administração, é possível visualizar todos os utilizadores registados, pesquisar, filtrar, editar dados, alterar níveis de acesso, bloquear ou eliminar contas.

1. Níveis de Utilizador

Os utilizadores estão classificados em três níveis distintos:
- 1: Comum – acesso básico às funcionalidades da aplicação.
- 2: Premium – acesso a funcionalidades extra (ex: receitas exclusivas, guardar favoritos, exportar planos).
- 3: Administrador – controlo total sobre utilizadores, categorias, receitas e sistema.

Atualmente, a distinção entre utilizadores comuns e premium ainda não está ativa no sistema, mas prevê-se no futuro que utilizadores premium possam:
- Marcar receitas como favoritas
- Exportar planos em PDF
- Aceder a filtros ou planos avançados
- Ver receitas exclusivas

2. Rota Flask – Listagem de Utilizadores (routes/admin.py)

```
@admin_bp.route("/utilizadores")
def listar_utilizadores():
        if session.get("user_nivel") != 3:
        flash("Acesso restrito a administradores.", "danger")
        return redirect(url_for("main.home"))
```

```
    utilizadores = listar_todos_utilizadores()
    form_apagar = ConfirmarAcaoForm()
    return render_template(
"admin/utilizadores.html",
    utilizadores=utilizadores,
    form_apagar=form_apagar
    )
```

3. Rota Flask – Edição de Utilizador (routes/admin.py)

```
@admin_bp.route("/utilizador/<int:id>/editar", methods=["GET", "POST"])
 def editar_utilizador(id):
    utilizador = obter_utilizador_por_id(id)
    if not utilizador:
    flash("Utilizador não encontrado.", "warning")
    return redirect(url_for("admin.listar_utilizadores"))

    form = EditarUserForm(obj=utilizador)
    if form.validate_on_submit():
    atualizar_dados_utilizador(
    id,
    novo_nome=form.nome.data,
    novo_email=form.email.data,
    novo_nivel=int(form.nivel.data),
    )
    flash("Utilizador atualizado com sucesso.", "success")
    return redirect(url_for("admin.ver_utilizador", id=id))

    form.nivel.data = str(utilizador.nivel)
    return render_template(
"admin/editar_utilizador.html", form=form, utilizador=utilizador
    )
```

4. Template HTML – Listagem Visual (templates/admin/utilizadores.html)

A interface apresenta os utilizadores com etiquetas visuais de nível e estado, estatísticas, e ações rápidas como 'Ver', 'Editar' e 'Bloquear'. Inclui também filtros por nível de acesso e estado da conta, e um campo de pesquisa por nome ou email.

Cada cartão de utilizador apresenta:
- Nome, email, e data de registo
- Número de receitas e planos criados
- Badges de nível (Administrador, Premium, Básico) e estado (Bloqueado)
- Ações: ver perfil, editar dados, bloquear/desbloquear.

Nota:

A funcionalidade de gestão de utilizadores é essencial para garantir a segurança, a personalização da experiência de utilização e a manutenção adequada do sistema. Apenas administradores têm acesso a estas operações, garantindo a proteção dos dados dos utilizadores e o controlo rigoroso sobre os privilégios atribuídos a cada conta.

Anexo X – Gestão de Categorias

Este anexo documenta a funcionalidade de gestão de categorias na aplicação Meal Planner. As categorias permitem classificar receitas e facilitar a sua pesquisa e organização dentro da plataforma. A gestão é feita no painel de administração, com possibilidade de criar, editar ou eliminar categorias, enquanto os utilizadores comuns podem visualizá-las e explorar as receitas associadas.

1. Rotas de Administração (routes/admin.py)

```
@admin_bp.route("/categorias")
 def gerir_categorias():
        categorias = listar_categorias()
        return render_template("admin/categorias.html", categorias=categorias,
form_apagar=FormApagarCategoria())

 @admin_bp.route("/categorias/criar", methods=["GET", "POST"])
 def criar_categoria_admin():
        form = CategoriaForm()
        if form.validate_on_submit():
        nova = criar_categoria(form.nome.data)
        flash("Categoria criada com sucesso!" if nova else "Já existe uma categoria com
esse nome.", "info")
      return redirect(url_for("admin.gerir_categorias"))
        return render_template("admin/criar_categoria.html", form=form)

 @admin_bp.route("/categorias/editar/<int:id>", methods=["GET", "POST"])
 def editar_categoria_admin(id):
        categoria = obter_categoria_por_id(id)
        form = CategoriaForm(obj=categoria)
        if form.validate_on_submit():
        editar_categoria(id, form.nome.data)
        flash("Categoria atualizada com sucesso.", "success")
        return redirect(url_for("admin.gerir_categorias"))
        return render_template("admin/editar_categoria.html", form=form,
categoria=categoria)

 @admin_bp.route("/categorias/apagar/<int:id>", methods=["POST"])
```

```
def eliminar_categoria_admin(id):
    if eliminar_categoria(id):
    flash("Categoria eliminada com sucesso.", "success")
    else:
    flash("A categoria está associada a receitas e não pode ser eliminada.", "danger")
    return redirect(url_for("admin.gerir_categorias"))
```

2. Visualização de Categorias (routes/categories.py)

```
@categories_bp.route("/")
def index():
    categorias = listar_categorias()
    return render_template("categories/index.html", categorias=categorias)

@categories_bp.route("/<int:categoria_id>")
def ver(categoria_id):
    categoria = obter_categoria_por_id(categoria_id)
    receitas = [r for r in listar_receitas() if r.categoria_id == categoria_id]
    return render_template("categories/ver.html", categoria=categoria, receitas=receitas)
```

3. Templates (categories/index.html e ver.html)

O template `index.html` apresenta as categorias em formato de grelha com ícones e botão 'Ver Receitas'. O template `ver.html` apresenta as receitas associadas a uma categoria específica, com detalhes como tempo, dificuldade, tags e botão de acesso ao detalhe.

Nota:

Esta estrutura permite ao administrador gerir de forma eficiente as categorias, assegurando a organização e manutenção das receitas na plataforma. Os utilizadores comuns beneficiam de uma navegação facilitada e de um acesso rápido às receitas organizadas por temas alimentares.